

FlashAttention: 具备IO感知的快速内存高效精确注意力

特里·道[†]、丹尼尔·Y·傅[†]、斯特凡诺·埃尔蒙[‡]、阿特里·鲁德拉[‡]和克里斯托弗·雷[†]

[†]斯坦福大学计算机科学系[‡]纽约州立大学布法罗分校计算机科学与工程系

{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

2022年6月24日

摘要

Transformer在处理长序列时速度缓慢且内存消耗大，因为自注意力机制的时间与内存复杂度随序列长度呈二次方增长。各类近似注意力方法尝试通过牺牲模型质量来降低计算复杂度以解决这一问题，但往往未能实现实际运行速度的提升。我们认为，缺失的关键原则是使注意力算法具备IO感知能力——即充分考虑GPU内存层级间的数据读写。我们提出FlashAttention，这是一种具备IO感知能力的精确注意力算法，它通过分块处理来减少GPU高带宽内存与GPU片上SRAM之间的内存读写次数。我们分析了FlashAttention的IO复杂度，证明其所需的HBM访问次数少于标准注意力机制，并在一定SRAM容量范围内具有最优性。我们还将FlashAttention扩展至块稀疏注意力，由此产生了一种比现有任何近似注意力方法都更快的近似注意力算法。FlashAttention训练Transformer的速度超越了现有基准：在BERT-large（序列长度512）上实现了15%的端到端实际加速，优于MLPerf 1.1训练速度纪录；在GPT-2（序列长度1K）上实现了3×倍加速；在长序列竞技场（序列长度1K-4K）上实现了2.4×倍加速。FlashAttention与块稀疏FlashAttention使Transformer能够处理更长的上下文，从而产生更高质量的模型（GPT-2的困惑度降低0.7，长文档分类任务提升6.4个点），并开启了全新能力：首次有Transformer在Path-X挑战（序列长度16K，准确率61.4%）和Path-256挑战（序列长度64K，准确率63.1%）上取得了优于随机猜测的表现。

1 引言

Transformer模型[82]已成为自然语言处理和图像分类等应用中最为广泛使用的架构。Transformer模型规模不断增大[5]、深度不断增加[83]，但为其配备更长的上下文仍面临困难[80]，因为其核心的自注意力模块的时间和内存复杂度与序列长度呈二次方关系。一个重要的问题是，使注意力机制更快、内存效率更高，能否帮助Transformer模型解决处理长序列时的运行时和内存挑战。

许多近似的注意力方法旨在降低注意力的计算和内存需求。这些方法涵盖范围广泛，从稀疏近似[51, 74]到低秩近似[12, 50, 84]，以及它们的组合方法[3, 9, 92]。尽管这些方法将计算需求降低到与序列长度成线性或近似线性的程度，但其中许多方法并未展现出相对于标准注意力的实际运行时间加速，因此未获得广泛采用。一个主要原因在于它们侧重于减少浮点运算次数（这可能与实际运行速度不相关），并倾向于忽略来自内存访问（输入输出）的开销。

在本文中，我们主张一个缺失的原则是使注意力算法具备IO感知能力[1]——即仔细考虑对不同层次快速和慢速内存的读写操作（例如，在快速的GPU片上SRAM与相对较慢的GPU高带宽内存HBM[45]之间，图1左）。在现代

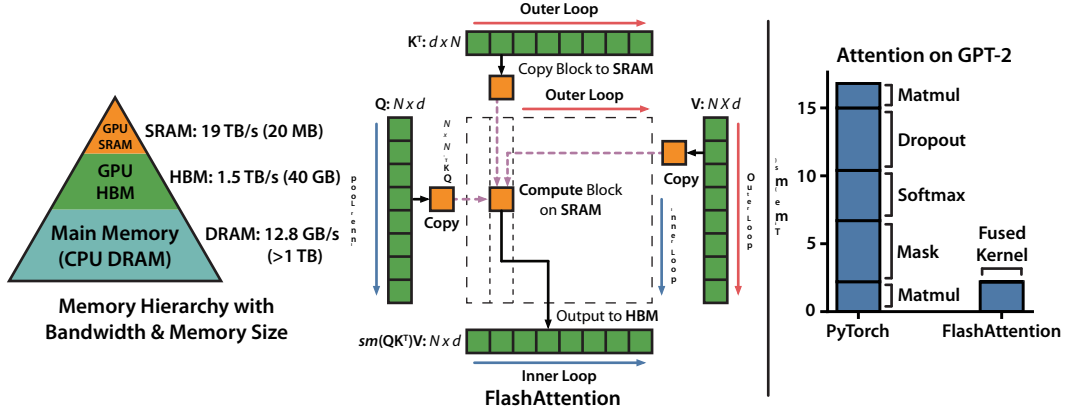


图1：左：FlashAttention采用分块处理来避免在（相对）较慢的GPU HBM上物化大型的 $\times$ 注意力矩阵（虚线框）。在外循环（红色箭头）中，FlashAttention遍历 K 和 V 矩阵的块，并将它们加载到快速的片上SRAM。在每个块内，FlashAttention遍历 Q 矩阵的块（蓝色箭头），将它们加载到SRAM，并将注意力计算的输出写回HBM。右：在GPT-2上相对于PyTorch注意力实现的加速比。FlashAttention不会将大型的 $\times$ 注意力矩阵读写到HBM，使得注意力计算实现了7.6 $\times$ 倍的加速。

GPU的计算速度已超越内存速度[61, 62, 63]，而Transformer中的大多数操作受限于内存访问[43]。对于类似的受内存限制的操作，即在读写数据可能占据运行时间很大一部分的情况下——例如数据库连接[71]、图像处理[70]、数值线性代数[4]及其他[40, 85]——IO感知算法至关重要。然而，常见的深度学习Python接口，如PyTorch和TensorFlow，并不允许对内存访问进行精细控制。

我们提出 FlashAttention，这是一种新型注意力算法，能够以极低的内存访问量精确计算注意力机制。我们的主要目标是避免从高带宽内存（HBM）中读取和写入注意力矩阵。这需要：（i）在不访问整个输入的情况下完成 softmax 归约计算；（ii）不为反向传播存储庞大的中间注意力矩阵。我们采用两项成熟技术来应对这些挑战：（i）重构注意力计算流程，将输入拆分为多个块并分块多次遍历，从而逐步完成 softmax 归约（即分块计算）；（ii）利用前向传播时存储的 softmax 归一化因子，在反向传播中于芯片内快速重算注意力，这比标准方法从 HBM 读取中间注意力矩阵更为高效。我们在 CUDA 中实现 FlashAttention，以精细化控制内存访问，并将所有注意力操作融合到一个 GPU 内核中。尽管重算带来额外的浮点运算量（FLOPs），但得益于 HBM 访问量大幅减少，我们的算法运行速度更快（在 GPT-2 上最高达 7.6 倍 [67]，见图 1 右）且内存占用更小——与标准注意力相比，内存消耗随序列长度线性增长。

我们分析了 FlashAttention 的 IO 复杂度 [1]，证明它需要 $(2^2 - 1)$ 次 HBM 访问，其中 2 是头维度， 2 是 SRAM 的大小，而标准注意力需要 $\Omega(2 + 2^2)$ 。对于典型的 2 和 2 值，FlashAttention 所需的 HBM 访问次数比标准注意力少很多倍（如图 2 所示，最多可减少 9 $\times$ 倍）。此外，我们给出了一个下界，证明在所有 SRAM 大小下，没有精确的注意力算法能在渐近意义上改进 HBM 访问次数。

我们还展示了 FlashAttention 可作为一种有用的原语，通过克服其内存访问开销的问题，来实现近似注意力算法的潜力。作为概念验证，我们实现了块稀疏 FlashAttention，这是一种稀疏注意力算法，其速度甚至达到 FlashAttention 的 2-4 $\times$ 倍，可扩展至序列长度 64k。我们证明，块稀疏 FlashAttention 的 IO 复杂度优于 FlashAttention，其提升倍数与稀疏率成比例。我们进一步探讨了扩展至其他操作（多 GPU 上的注意力、核回归、块稀疏矩阵）的应用。

乘法)在第5节中。我们开源了FlashAttention，以便更容易地基于这个原语进行构建。¹

我们通过实验验证了 FlashAttention 通过建模更长上下文来加速模型训练并提升模型质量。我们还对 FlashAttention 和块稀疏 FlashAttention 的运行时间及内存占用进行了基准测试，并与先前的注意力机制实现进行了比较。

- 更快的模型训练。FlashAttention 在挂钟时间内更快地训练 Transformer 模型。我们训练 BERT-large（序列长度 512）的速度比 MLPerf 1.1 [58] 中的训练速度记录快 15%，训练 GPT2（序列长度 1K）的速度比 HuggingFace [87] 和 Megatron-LM [77] 的基线实现快 3×，训练 long-range arena（序列长度 1K-4K）的速度比基线快 2.4×。
- 更高质量的模型。FlashAttention 可将 Transformer 扩展至更长序列，从而提升模型质量并解锁新能力。我们在 GPT-2 上观察到困惑度改善 0.7，而在长文档分类任务中通过建模更长序列获得 6.4 个点的提升 [13]。Flash Attention 使得首个 Transformer 仅凭使用更长序列长度（16K）就能在 Path-X [80] 挑战中取得超越随机水平的表现。块稀疏 FlashAttention 则让 Transformer 能够扩展到更长的序列（64K），诞生了首个能在 Path-256 上实现超越随机水平表现的模型。
- 注意力基准测试。FlashAttention 在常见序列长度（128 到 2K）上比标准注意力实现快最多 3×，并可扩展至 64K。在序列长度不超过 512 时，FlashAttention 比任何现有注意力方法都更快且内存效率更高，而对于超过 1K 的序列长度，一些近似注意力方法（例如 Linformer）开始变得更快。另一方面，块稀疏 FlashAttention 比我们所知的所有现有近似注意力方法都快。

2 背景

我们提供了一些关于现代硬件（GPU）上常见深度学习操作的性能特征背景。同时，我们也介绍了注意力的标准实现方式。

2.1 硬件性能

我们在此专注于 GPU。其他硬件加速器的性能相似 [46, 48]。

GPU 内存层次结构。GPU 内存层次结构（图 1 左侧）包含多种不同大小与速度的内存形式，其中容量越小的内存速度越快。以 A100 GPU 为例，其配备 40-80 吉字节（GB）的高带宽内存（HBM），带宽达 1.5-2.0 太字节/秒（TB/s），同时每个流式多处理器（共 108 个）都配有 192 千字节（KB）的片上 SRAM，估计带宽约 1.9 太字节/秒 [44, 45]。片上 SRAM 的速度比 HBM 快一个数量级，但容量却要小几个数量级。由于计算速度相对于内存速度提升更快 [61, 62, 63]，运算越来越受限于内存（HBM）的访问瓶颈。因此，充分利用高速 SRAM 变得愈发重要。

执行模型。GPU 拥有大量线程来执行被称为内核的操作。每个内核将输入从 HBM 加载到寄存器和 SRAM，进行计算，然后将输出写回 HBM。

性能特性方面，根据计算与内存访问的平衡程度，操作可分为计算受限型或内存受限型。这通常通过算术强度 [85] 来衡量，即每字节内存访问所执行的算术运算次数。

1. 计算受限：操作耗时主要由算术运算的数量决定，而访问 HBM 的时间则要短得多。典型例子包括大内维度的矩阵乘法，以及具有大量通道数的卷积运算。

2. 内存受限：操作所需时间由内存访问次数决定，而计算所花的时间则少得多。例子包括大多数其他操作：逐元素操作（例如，激活函数、dropout），以及归约操作（例如，求和、softmax、批归一化、层归一化）。

核融合。加速内存受限操作最常见的方法是核融合：如果对同一输入应用多个操作，则可以从 HBM 加载一次输入，而不必为每个操作加载多次。编译器可以自动融合许多逐元素操作 [53, 65, 75]。

¹FLASHATTENTION code is available at <https://github.com/HazyResearch/flash-attention>

然而，在模型训练的背景下，中间值仍需写入HBM以保存用于反向传递，这降低了朴素内核融合的有效性。

2.2 标准注意力实现

给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，其中 N 是序列长度， d 是头维度，我们希望计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 softmax 是按行应用的。

标准注意力实现会将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 显式存入 HBM，这需要 (N^2) 的内存。通常 $N \gg d$ （例如 GPT2 中， $N = 1024$ ， $d = 64$ ）。算法 0 中描述了标准的注意力实现。由于部分甚至大部分操作受限于内存带宽（如 softmax ），大量的内存访问导致实际运行时间变慢。

其他针对注意力矩阵的逐元素操作进一步加剧了这一问题，例如应用于 $\mathbf{S}$ 的掩码或应用于 $\mathbf{P}$ 的丢弃。因此，业界进行了诸多尝试以将多个逐元素操作融合，例如将掩码与 softmax 融合[77]。

在第 3.2 节中，我们将展示标准注意力实现的 HBM 访问次数与序列长度 N 呈二次方关系。我们还将比较标准注意力机制与我们提出的 FlashAttention 方法在浮点运算次数和 HBM 访问次数上的差异。

算法 0 标准注意力实现

要求：矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 在 HBM 中。

- 1: 按块从 HBM 加载 $\mathbf{Q}, \mathbf{K}$ ，计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$ ，将 $\mathbf{S}$ 写入 HBM。
 - 2: 从 HBM 读取 $\mathbf{S}$ ，计算 $\mathbf{P} = \text{softmax}(\mathbf{S})$ ，将 $\mathbf{P}$ 写入 HBM。
 - 3: 按块从 HBM 加载 $\mathbf{P}$ 和 $\mathbf{V}$ ，计算 $\mathbf{O} = \mathbf{P}\mathbf{V}$ ，将 $\mathbf{O}$ 写入 HBM。
 - 4: 返回 $\mathbf{O}$ 。
-

3 FlashAttention：算法、分析与扩展

我们展示了如何在减少 HBM 读取/写入次数且无需为反向传播存储大型中间矩阵的情况下精确计算注意力。由此产生的注意力算法既内存高效，又在实际时钟时间上更快。我们分析了其 IO 复杂度，表明我们的方法相比标准注意力需要远少的 HBM 访问。我们进一步展示，FlashAttention 可通过扩展来处理块稀疏注意力，从而作为一个有用的原语发挥作用。

为了便于说明，本文重点关注前向传递；附录 B 包含反向传递的详细信息。

3.1 一种结合分块和重计算的高效注意力算法

给定在 HBM 中的输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，我们旨在计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 并将其写入 HBM。我们的目标是减少 HBM 访问次数（对于 N 达到次二次方级别）。

我们采用两种成熟技术（分块、重计算）来克服在亚二次方 HBM 访问次数下计算精确注意力的技术挑战。我们在算法 1 中对此进行了描述。核心思路是将输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 分割成块，从慢速 HBM 加载到快速 SRAM 中，然后针对这些块计算注意力输出。通过在相加之前用正确的归一化因子对每个块的输出进行缩放，我们最终得到了正确的结果。

分块。我们按块计算注意力。Softmax 将 $\mathbf{K}$ 的列耦合，因此我们使用缩放来分解大型 softmax [51, 60, 66]。为数值稳定性，向量 $\mathbf{x} \in \mathbb{R}^B$ 的 softmax 计算如下：

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}], \quad \ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

对于向量 $(1), (2) \in \mathbb{R}^B$ ，我们可以将拼接后的 $\begin{bmatrix} (1) & (2) \end{bmatrix} \in \mathbb{R}^{2B}$ 的 softmax 分解为：

$$m(x) = m(\begin{bmatrix} x^{(1)} & x^{(2)} \end{bmatrix}) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \begin{bmatrix} e^{m(x^{(1)})-m(x)} f(x^{(1)}) & e^{m(x^{(2)})-m(x)} f(x^{(2)}) \end{bmatrix},$$

$$\ell(x) = \ell(\begin{bmatrix} x^{(1)} & x^{(2)} \end{bmatrix}) = e^{m(x^{(1)})-m(x)} \ell(x^{(1)}) + e^{m(x^{(2)})-m(x)} \ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

因此，如果我们跟踪一些额外的统计量 $(\cdot), \ell(\cdot)$ ，就可以逐步计算 softmax。² 于是我们将输入 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 分块（算法1第3行），计算 softmax 值及其额外统计量（算法1第10行），然后合并结果（算法1第12行）。

重计算。我们的目标之一是不存储反向传播所需的 $(\cdot)$ 与 $(\cdot)$ 个中间值。反向传播通常需要矩阵 $\mathbf{S}$ 、 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 来计算相对于 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 的梯度。然而，通过存储输出 $\mathbf{O}$ 和 softmax 归一化统计量 $(\cdot), \ell(\cdot)$ ，我们可以在反向传播中从 SRAM 中的 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 块轻松重算注意力矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 。这可以看作一种选择性梯度检查点 [10, 34] 的形式。尽管梯度检查点已被提出用于减少最大内存需求 [66]，但所有（我们所知的）实现都不得不在速度与内存之间权衡。相比之下，即便 FLOPs 更多，由于减少了 HBM 访问，我们的重计算仍加速了反向传播（图 2）。完整的反向传播描述见附录 B。

实现细节：内核融合。分块（Tiling）使我们能够在单个 CUDA 内核中实现算法，从 HBM 加载输入，执行所有计算步骤（矩阵乘法、softmax，可选的掩码和 Dropout，矩阵乘法），然后将结果写回 HBM（掩码和 Dropout 详见附录 B）。这避免了反复从 HBM 读取和写入输入与输出。

算法 1 FlashAttention

要求：矩阵 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V} \in \mathbb{R}^{N \times d}$ 存储在 HBM 中，片上 SRAM 大小为 $\text{SRAM}_{\text{tile}}$ 。1: 设置块大小 $c = \lceil \frac{M}{4d} \rceil$ 、 $r = \min(\lceil \frac{M}{4d} \rceil, \lfloor \frac{N}{B_r} \rfloor)$ 。2: 在 HBM 中初始化 $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$ 、 $\ell = (0)_N \in \mathbb{R}^N$ 、 $\tilde{\ell} = (-\infty)_N \in \mathbb{R}^N$ 。3: 将 $\mathbf{Q}$ 划分为 $r = \lfloor \frac{N}{B_r} \rfloor$ 个大小为 $r \times \times$ 的块 $\mathbf{Q}_1, \dots, \mathbf{Q}_r$ ，并将 $\mathbf{K}$ 和 $\mathbf{V}$ 分别划分为 $c = \lfloor \frac{N}{B_c} \rfloor$ 个大小为 $c \times \times$ 的块 $\mathbf{K}_1, \dots, \mathbf{K}_c$ 和 $\mathbf{V}_1, \dots, \mathbf{V}_c$ 。4: 将 $\mathbf{O}$ 划分为 r 个大小为 $r \times \times$ 的块 $\mathbf{O}_1, \dots, \mathbf{O}_r$ ，将 ℓ 划分为 r 个大小为 r 的块 $\ell_1, \dots, \ell_r$ ，将 $\tilde{\ell}$ 划分为 r 个大小为 r 的块 $\tilde{\ell}_1, \dots, \tilde{\ell}_r$ 。5: for $1 \leq i \leq c$ do 6: 将 $\mathbf{K}_i$ 、 $\mathbf{V}_i$ 从 HBM 加载到片上 SRAM。7: for $1 \leq j \leq r$ do 8: 将 $\mathbf{Q}_j$ 、 $\mathbf{O}_j$ 、 ℓ_j 、 $\tilde{\ell}_j$ 从 HBM 加载到片上 SRAM。9: 在片上计算 $\mathbf{S}_{ij} = \mathbf{Q}_j \mathbf{K}_i^T \in \mathbb{R}^{B_r \times B_c}$ 。10: 在片上计算 $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$ 、 $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise)、 $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$ 。11: 在片上计算 $\ell_i^{\text{new}} = \max(\ell_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$ 、 $\ell_i^{\text{new}} = m_i - m_i^{\text{new}} \ell_i + \tilde{m}_{ij} - m_i^{\text{new}} \tilde{\ell}_{ij}$ 。12: 将 $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) m_i - m_i^{\text{new}} \mathbf{O}_i + \tilde{m}_{ij} - m_i^{\text{new}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ 写回 HBM。13: 将 $\ell_i \leftarrow \ell_i^{\text{new}}$ 、 $\tilde{\ell}_i \leftarrow \ell_i^{\text{new}}$ 写回 HBM。14: end for 15: end for 16: 返回 $\mathbf{O}$ 。

我们展示了 FlashAttention 的正确性、运行时间和内存需求（证明见附录 C）。

定理 1. 算法 1 返回 $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^T)\mathbf{V}$ ，需要 $\mathcal{O}(N^2)$ FLOPs，且除输入输出外还需 $\mathcal{O}(N)$ 的额外内存。

3.2 分析：FlashAttention 的 IO 复杂性

我们分析了 FlashAttention 的 IO 复杂度，表明与标准注意力相比，其 HBM 访问次数显著减少。我们还给出了一个下界，证明没有任何精确注意力算法能够

²This style of aggregation is called *algebraic aggregation* [33].

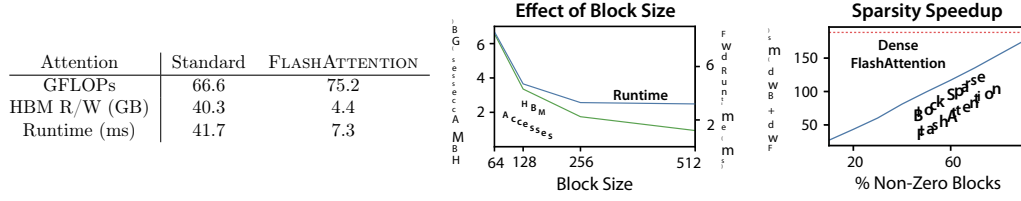


图2：左：在A100 GPU上，标准注意力与FlashAttention对GPT-2 medium（序列长度1024，头维度64，16个头，批量大小64）的前向+后向运行时间。HBM访问是影响运行时间的主要因素。中：在A100 GPU上，FlashAttention的前向运行时间（序列长度1024，头维度64，16个头，批量大小64）。更少的HBM访问导致更快的运行时间，直至某一临界点。右：块稀疏FlashAttention的运行时间（序列长度4K）比FlashAttention快，其加速比与稀疏度成正比。

在所有 SRAM 大小下，渐近地改善了 HBM 访问次数。证明见附录 C。

定理2. 设 L 为序列长度， d 为注意力头维度， B 为RAM的大小，且满足 $d \leq B \leq L$ 。标准注意力（算法）需进行 $\Theta(L^2 + B^2)$ 次HBM访问，而FlashAttention（算法1）仅需 $\Theta(L^2 B^{-1})$ 次HBM访问。

对于典型的 d 值（64-128）和 B （约100KB）， B^2 比 L^2 小许多倍，因此 FlashAttention 所需的 HBM 访问次数远少于标准实现。这带来了更快的执行速度和更低的内存占用，我们在第 4.3 节中验证了这一点。

证明的主要思想是，给定SRAM大小为 B ，我们可以加载大小为 $\Theta(B)$ 的K和V的块（算法1第6行）。对于每个K和V的块，我们遍历Q（算法1第8行）的所有块以计算中间值，导致对Q进行 $\Theta(L/B)$ 次遍历。每次遍历加载 $\Theta(B)$ 个元素，这相当于 $\Theta(L^2 B^{-1})$ 次HBM访问。我们类似地证明，标准注意力的反向传播需要 $\Theta(L^2 + B^2)$ 次HBM访问，而FlashAttention的反向传播需要 $\Theta(L^2 B^{-1})$ 次HBM访问（附录B）。

我们证明了一个下界：在计算精确注意力时，对于所有 B （SRAM 大小）的值，无法在渐近意义上改进 HBM 访问次数。

命题3. 设 L 为序列长度， d 为头维度， B 为RAM的大小，且满足 $d \leq B \leq L$ 。不存在一种算法能够对所有在范围 $[L/B, L]$ 内的 B ，仅使用 $\Theta(L^2 B^{-1})$ 次HBM访问即计算出精确注意力。

该证明基于以下事实：对于 $B = \Theta(L)$ 之间的 B ，任何算法都必须执行 $\Omega(L^2/B)$ 次HBM访问。这种针对 B 子区间的下界类型在流算法文献中很常见 [88]。我们将在未来的激动人心的工作中，证明以 B 为参数的参数化复杂度 [27] 下界。

我们验证了HBM访问次数是注意力运行时的主要决定因素。在图2（左）中，我们看到尽管FlashAttention的FLOP计数相比标准注意力更高（由于反向传播中的重计算），但其HBM访问次数少得多，从而运行时快得多。在图2（中）中，我们变化FlashAttention的块大小 B_c ，这会带来不同的HBM访问量，并测量前向传播的运行时间。随着块大小增加，HBM访问次数减少（因为对输入的遍历次数减少），运行时也减少。对于足够大的块大小（超过256），运行时则受限于其他因素（例如算术运算）。此外，更大的块大小将无法容纳在较小的SRAM中。

3.3 扩展：块稀疏 FlashAttention

我们将FlashAttention扩展为近似注意力机制：我们提出了块稀疏FlashAttention，其IO复杂度相比FlashAttention降低的倍数与稀疏度成比例。

给定输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 和一个掩码矩阵 $\tilde{\mathbf{M}} \in \{0, 1\}^{N \times N}$ ，我们要计算：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S} \odot \tilde{\mathbf{M}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 $(\mathbf{S} \odot \tilde{\mathbf{M}})_{kl} = S_{kl}$ 若 $\tilde{M}_{kl} = 1$ 且 $-\infty$ 若 $\tilde{M}_{kl} = 0$ 。我们要求 $\tilde{\mathbf{M}}$ 具有分块形式：对于某些块大小 r, c ，对所有 k, l ， $\tilde{M}_{k,l} = M_{i,j}$ 有 $i = \lfloor k/r \rfloor$ ， $j = \lfloor l/c \rfloor$ 对某个 $\mathbf{M} \in \{0, 1\}^{N/B_r \times N/B_c}$ 。

给定预定义的块稀疏掩码 $M \in \{0, 1\}^{N/B_r \times N/B_c}$ ，我们可以轻松地调整算法1，使其仅计算注意力矩阵的非零块。该算法与算法1相同，只是会跳过零块。我们在附录B的算法5中重现了该算法的描述。

我们还分析了块稀疏FlashAttention的IO复杂度。

命题 4. 设 L 为序列长度， d 为头维度，且 B 为 SRAM 的大小，满足 $B_r \leq B \leq B_c$ 。块稀疏FlashAttention (算法 5) 需要 $\Theta(L^2 + \frac{L^2}{B} \log \frac{L}{B})$ 次 HBM 访问，其中 α 是块稀疏掩码中非零块的比例。

我们看到，应用块稀疏性通过稀疏性直接改进了IO复杂度中的较大项。对于较大的序列长度 L ，通常设置为 $L^{-1/2}$ [11]或 $L^{-1} \log L$ [3, 17, 92]，从而得到 $\Theta(\sqrt{L})$ 或 $\Theta(\log L)$ 的IO复杂度。在下游实验中，我们使用固定的蝴蝶稀疏模式[17]，该模式已被证明能够近似任意稀疏性[16]。

在图2（右）中，我们验证了随着稀疏度的增加，块稀疏FlashAttention的运行时间成比例地缩短。在LRA基准测试上，块稀疏FlashAttention实现了2.8×倍的加速，同时性能与标准注意力相当（第4节）。

4 实验

我们评估使用FlashAttention训练Transformer模型的影响。我们验证关于训练时间和模型准确性的两个说法，并报告注意力运行时间和内存基准。

- 训练速度。FlashAttention 在 BERT 上比 MLPerf 1.1 [58] 的速度记录提升了 15%，并且相对于 HuggingFace [87] 将 GPT-2 的速度提升了高达 3×，相对于 Megatron [77] 的标准 Transformer 提升了 1.8×。FlashAttention 将长程竞技场 (LRA) 基准测试的速度提升了 2.4×。
- 质量。FlashAttention 将 Transformer 扩展到更长的序列，从而获得更高的质量。FlashAttention 训练上下文长度为 4K 的 GPT-2，比 Megatron 训练上下文长度为 1K 的 GPT-2 更快，同时困惑度改善了 0.7。对更长的序列进行建模，在两个长文档分类任务上带来了 6.4 个百分点的提升。最后，FlashAttention 诞生了首个能在具有挑战性的 Path-X 任务（序列长度 16K）上实现优于随机性能的 Transformer，而块稀疏 FlashAttention 则诞生了我们所知的首个能在 Path-256（序列长度 64K）上实现优于随机性能的序列模型。
- 注意力机制基准测试。我们基于序列长度评估了FlashAttention和块稀疏FlashAttention的运行时间和内存性能。我们确认，FlashAttention的内存占用量与序列长度呈线性关系，并且在常见序列长度（最高2K）下，其速度比标准注意力快最多3×。我们确认，块稀疏FlashAttention的运行时间与序列长度呈线性关系，并且比所有现有的近似注意力基线更快。更多实验细节见附录E。

4.1 利用FlashAttention加速模型

BERT。FlashAttention 实现了我们所知最快的单节点 BERT 训练速度。我们使用 FlashAttention 在 Wikipedia 上训练了一个 BERT-large [22] 模型。表 1 将我们的训练时间与 Nvidia 为 MLPerf 1.1 [58] 创下训练速度纪录的实现进行了比较。我们的实现速度提升了 15%。

表1：BERT-large的训练时间，从MLPerf基准提供的相同初始化开始，达到掩码语言建模72.0%的目标准确率。在8×A100 GPU上平均10次运行。

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FLASHATTENTION (ours)	17.4 ± 1.4

GPT-2。FlashAttention在大型OpenWebtext数据集[32]上为GPT-2 [67]带来了比广泛使用的HuggingFace [87]和Megatron-LM [77]实现更快的训练时间。表2显示，与Huggingface相比，端到端加速高达3×倍，与Megatron-LM相比，加速达1.7×倍。FlashAttention

达到了与其他两种实现相同的困惑度，因为我们没有改变模型定义。附录E包含了训练过程中验证困惑度的曲线图，证实了FlashAttention与基线方法在数值上同样稳定，并产生了相同的训练/验证曲线。表2：使用FlashAttention的GPT-2小型和中型模型，与Huggingface实现相比速度提升最高可达3×倍，与Megatron-LM相比最高可达1.7×倍。训练时间基于8块×A100 GPU报告。

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	2.7 days (3.5×)
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	6.9 days (3.0×)

长距离竞技场。我们在长距离竞技场（LRA [80]）基准测试上比较了原始Transformer（采用标准实现或Flash Attention）。测量了所有模型的准确率、吞吐量和训练时间。每个任务具有不同的序列长度，在1024到4096之间变化。我们遵循Tay等人[80]和Xiong等人[90]的实现与实验设置。³表3显示，与标准注意力相比，FlashAttention实现了高达2.4×的加速。块稀疏FlashAttention比我们测试过的所有近似注意力方法都要快。

表3：标准注意力、FlashAttention、块稀疏FlashAttention以及近似注意力基线在Long-Range-Arena基准测试上的表现。

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FLASHATTENTION	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FLASHATTENTION	37.0	63.0	81.3	43.6	73.3	59.6	2.8×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 更长序列的更好模型

长上下文语言建模。FlashAttention 的运行速度和内存效率使我们能将 GPT-2 的上下文长度增加 4×，同时仍比 Megatron-LM 的优化实现更快。表 4 显示，采用 FlashAttention 且上下文长度为 4K 的 GPT-2，速度仍比 Megatron 上下文长度为 1K 的 GPT-2 快 30%，同时困惑度降低 0.7。

表4：使用FlashAttention的GPT-2 small，在上下文长度较Megatron-LM大4×倍的情况下，仍然快30%，同时困惑度改善了0.7。报告了在8×块A100 GPU上的训练时间。

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FLASHATTENTION	1k	18.2	2.7 days (1.7×)
GPT-2 small - FLASHATTENTION	2k	17.6	3.0 days (1.6×)
GPT-2 small - FLASHATTENTION	4k	17.5	3.6 days (1.3×)

长文档分类。使用 FlashAttention 训练具有更长序列的 Transformer 提高了在 MIMIC-III [47] 和 ECtHR [6, 7] 数据集上的性能。MIMIC-III 包含重症监护室患者的出院摘要，每份均标注多个标签。ECtHR 包含来自

³LRA accuracy results are known to be highly dependent on the tuning procedure [90]. Our reproduced baselines perform better than as reported in the original comparison [80].

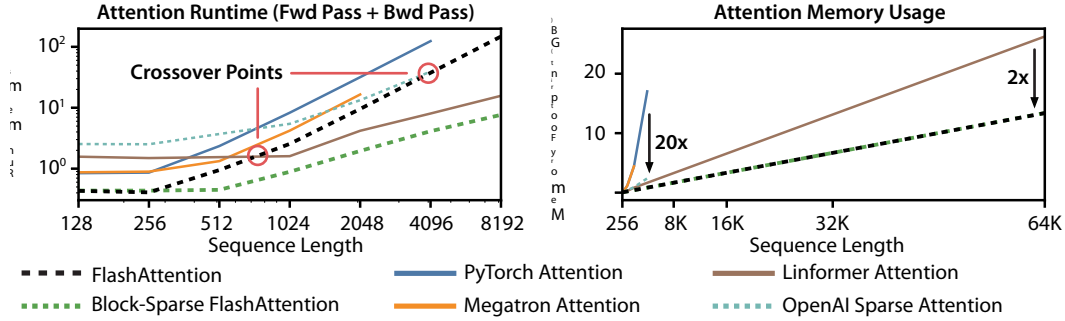


图3：左侧：前向传播 + 反向传播的运行时间。右侧：注意力内存使用量。

欧洲人权法院，每个案例都对应到据称遭到违反的《人权公约》条款。这两个数据集均包含非常长的文本文档；MIMIC 中的平均词元数为 2,395 个，最长文档包含 14,562 个词元，而 ECtHR 中的平均值和最长值分别为 2,197 和 49,392。我们评估了增加预训练 RoBERTa 模型 [56] 的序列长度所带来的提升（我们按照 Beltagy 等人 [3] 的做法重复位置嵌入）。

表5显示，在MIMIC数据集上，序列长度16K比长度512高出4.3分，而在ECtHR数据集上，长度8K比长度512高出8.5分。这种差异可能是由于细微的分布偏移：MIMIC-III包含专业医学文本，因此可能更容易受到文档长度分布偏移的影响，而ECtHR则包含通用语言。

表5：使用FlashAttention在不同序列长度下的长文档性能（micro ₁）

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	57.1
ECtHR [6]	72.2	74.3	77.1	78.6	80.7	79.2

表6：我们报告了首个在Path-X和Path-256上达到非随机性能的Transformer模型。

Model	Path-X	Path-256
Transformer	X	X
Linformer [84]	X	X
Linear Attention [50]	X	X
Performer [12]	X	X
Local Attention [80]	X	X
Reformer [51]	X	X
SMYRF [19]	X	X
FLASHATTENTION	61.4	X
Block-sparse FLASHATTENTION	56.0	63.1

路径-X 和 路径-256。路径-X 和路径-256 基准是来自长程竞技场（long-range arena）基准中的挑战性任务，旨在测试长上下文能力。任务要求判断一张 128×128（或 256×256）的黑白图像中两点之间是否存在一条连接路径，图像以每次一个像素的方式输入 Transformer。在先前工作中，所有 Transformer 模型要么内存耗尽，要么仅能取得随机水平的性能[80]。因此，人们一直在寻找能够建模这种长上下文的替代架构[37]。我们在此首次呈献 Transformer 模型成功求解路径-X 和路径-256 的结果（表 6）。我们在路径-64 上预训练 Transformer，然后通过对位置嵌入进行空间插值迁移至路径-X。FlashAttention 在路径-X 上达到了 61.4 的准确率。此外，块稀疏 FlashAttention 使 Transformer 能够扩展到 64K 序列长度，在路径-256 上取得了 63.1 的准确率⁴。

4.3 注意力基准测试

我们变化序列长度，在配备40 GB HBM的单一A100 GPU上，使用dropout和填充掩码，测量FlashAttention和块稀疏FlashAttention相对于各种注意力基线方法的运行时间和内存使用情况。我们与精确注意力、近似注意力和稀疏注意力的参考实现进行了对比。正文部分报告了部分基线方法；附录E包含更多基线方法及完整细节。

⁴Path-256需要更长的序列，但路径相对更短，因此更容易获得更高的准确率。

运行时间。图3（左）报告了FlashAttention与块稀疏FlashAttention的前向+反向传播运行时间（以毫秒计），并与精确、近似及稀疏注意力基线方法进行了对比（精确数值见附录E）。运行时间随序列长度呈二次增长，但FlashAttention的运行速度显著快于精确注意力基线，比PyTorch实现快至3×倍。许多近似/稀疏注意力机制的运行时间随序列长度线性增长，但对于短序列，FlashAttention由于更少的内存访问仍比近似和稀疏注意力更快。在序列长度为512至1024之间时，近似注意力的运行时间开始与FlashAttention出现交叉。另一方面，在所有序列长度下，块稀疏FlashAttention都比我们所知的所有精确、稀疏及近似注意力实现更快。

内存占用。图3（右）显示了FlashAttention和块稀疏FlashAttention与各种精确、近似和稀疏注意力基线的内存占用对比。FlashAttention和块稀疏FlashAttention具有相同的内存占用，二者均随序列长度线性增长。FlashAttention的内存效率最高比精确注意力基线高出20×倍，且比近似注意力基线更节省内存。除Linformer外，所有其他算法在A100 GPU上均于序列长度达到64K前耗尽内存，而FlashAttention仍然比Linformer高效2×倍。

5 局限性与未来方向

我们讨论了我们方法的局限性和未来方向。相关工作请参见附录A。

编译至CUDA。我们当前构建IO感知注意力实现的方法，需要为每个新的注意力实现编写新的CUDA内核。这要求使用比PyTorch低级得多的语言来编写注意力算法，并且需要大量的工程投入。实现方式也可能无法在不同GPU架构之间迁移。这些局限性表明，需要一种支持用高级语言（例如PyTorch）编写注意力算法，并将其编译为CUDA中IO感知实现的方法——类似于图像处理领域Halide所做的工作[70]。

IO感知深度学习。我们认为IO感知的方法可以扩展到注意力之外。注意力是Transformer中内存最密集的计算，但深度网络中的每一层都会触及GPU HBM。我们希望我们的工作能启发其他模块的IO感知实现。我们在附录D中讨论了这些潜在的扩展。

多GPU IO感知方法。我们的IO感知注意力实现在单个GPU上计算注意力时，在常数因子内是最优的。然而，注意力计算可以在多个GPU上并行化[72]。使用多个GPU为IO分析增加了一个额外的层次——需要考虑GPU之间的数据传输。我们希望我们的工作能激发未来在这个方向上的研究。

致谢

我们的实现以Apex的FMHA代码（<https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha>）为起点。我们感谢Young-Jun Ko深入讲解他的FMHA实现，并耐心回答我们关于CUDA的问题。感谢Sabri Euboglu、Megan Leszczynski、Laurel Orr、Yuhuai Wu、Beidi Chen和Xun Huang对本文早期草稿提出的建设性反馈与建议。我们感谢Markus Rabe和Charles Staats就他们的注意力算法进行的富有帮助的讨论。

我们衷心感谢以下机构的支持：NIH（美国国立卫生研究院）项目编号U54EB020405（Mobilize）、NSF（美国国家科学基金会）项目编号CCF1763315（超越稀疏性）、CCF1563078（从容量到速度）及1937301（RTML）；ARL（美国陆军研究实验室）项目编号W911NF-21-2-0251（人机交互式AI协作）；ONR（美国海军研究办公室）项目编号N000141712266（统一弱监督）、N00014-20-1-2480（理解并应用机器学习中的非欧几里得几何）及N000142012275（NEPTUNE）；以及NXP、Xilinx、LETI-CEA、Intel、IBM、Microsoft、NEC、Toshiba、TSMC、ARM、Hitachi、BASF、Accenture、Ericsson、Qualcomm、Analog Devices、Google Cloud、Salesforce、Total等企业，HAI-GCP与HAI-Azure云研究学分计划，斯坦福数据科学计划（SDSI），美国国防部通过国防科学与工程研究生奖学金（NDSEG）计划提供的资助，以及斯坦福DAWN项目成员：Facebook、Google和VMWare。美国政府被授权为政府目的的复制和分发本重印文件。

尽管其上存在版权标记。本材料所表达的任何观点、发现、结论或建议均为作者个人见解，并不一定反映美国国立卫生研究院、美国海军研究办公室或美国政府的观点、政策或认可，无论明示或暗示。Atri Rudra的研究得到了美国国家科学基金会拨款CCF-1763481的部分支持。

参考文献

- [1] Alok Aggarwal 和 Jeffrey S Vitter。排序及相关问题的输入/输出复杂性。《美国计算机学会通讯》，31(9):1116–1127，1988年。
- [2] Irwan Bello. LambdaNetworks：无需注意力机制的长程交互建模. arXiv预印本 arXiv:2102.08602, 2021.
- [3] 伊兹·贝尔塔吉、马修·E彼得斯与阿尔曼·科汉。Longformer：长文档转换器。arXiv预印本，编号arXiv:2004.05150，2020年。
- [4] L Susan Blackford, Antoine Petit, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, 等. 一套更新的基础线性代数子程序 (blas) . ACM数学软件汇刊, 28(2):135–151, 2002.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, 等. 语言模型是少样本学习器. 神经信息处理系统进展, 33:1877–1901, 2020.
- [6] Ilias Chalkidis, Ion Androutsopoulos, 和 Nikolaos Aletras. 基于神经网络的英文法律判决预测. 见第57届计算语言学协会年会论文集, 第4317–4323页, 意大利佛罗伦萨, 2019. 计算语言学协会. doi: 10.18653/v1/P19-1424. URL <https://www.aclweb.org/anthology/P19-1424>.
- [7] Ilias Chalkidis, Manos Fergadiotis, Dimitrios Tsarapatsanis, Nikolaos Aletras, Ion Androutsopoulos 与 Prodromos Malakasiotis. 通过正则化的段落级理由提取：欧洲人权法院案例研究. 见计算语言学协会北美分会年度会议论文集, 墨西哥城, 墨西哥, 2021年. 计算语言学协会.
- [8] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, 和 Ghislain Durif. 在GPU上的核运算，具备自动微分功能，且无内存溢出. 机器学习研究杂志, 22(74):1–6, 2021. URL <http://jmlr.org/papers/v22/20-275.html>.
- [9] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra 和 Christopher Ré. Scatterbrain：统一稀疏与低秩注意力机制. 发表于《神经信息处理系统大会进展》(NeurIPS), 2021.
- [10] Tianqi Chen, Bing Xu, Chiyuan Zhang, 和 Carlos Guestrin. 训练具有次线性内存成本的深度网络. arXiv preprint arXiv:1604.06174, 2016.
- [11] Rewon Child, Scott Gray, Alec Radford 和 Ilya Sutskever. 《使用稀疏Transformer生成长序列》。arXiv预印本 arXiv:1904.10509, 2019年。
- [12] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, 等. 使用Performers重新思考注意力机制. 收录于国际学习表征会议(ICLR), 2020.
- [13] Xiang Dai, Ilias Chalkidis, Sune Darkner, Desmond Elliott. 重新审视基于Transformer的长文档分类模型. arXiv preprint arXiv:2204.06683, 2022.
- [14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL：超越固定长度上下文的注意力语言模型. 见第57届计算语言学协会年会会议录, 页码2978–2988, 2019.

- [15] Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra 和 Christopher Ré. 利用蝴蝶分解学习线性变换的快速算法。收录于国际机器学习会议 (ICML), 2019。
- [16] Tri Dao, Nimit Sohoni, Albert Gu, Matthew Eichhorn, Amit Blonder, Megan Leszczynski, Atri Rudra, 和 Christopher Ré. Kaleidoscope: 所有结构化线性映射的高效、可学习表示。国际学习表征会议 (ICLR), 2020。
- [17] Tri Dao, Beidi Chen, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. 像素化蝴蝶：神经网络模型的简单高效稀疏训练。In International Conference on Learning Representations (ICLR), 2022。
- [18] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: 用于高效准确训练的表达式结构化矩阵。载于国际机器学习大会 (ICML), 2022。
- [19] Giannis Daras, Nikita Kitaev, Augustus Odena, 和 Alexandros G Dimakis. 使用不对称聚类的Smyrf高效注意力机制。《神经信息处理系统进展》, 33:6476–6489, 2020。
- [20] Christopher De Sa, Albert Gu, Rohan Puttagunta, Christopher Ré 和 Atri Rudra. 结构化稠密矩阵向量乘法的双管齐下进展。收录于《第二十九年度ACM-SIAM离散算法研讨会论文集》, 第1060–1079页。SIAM, 2018。
- [21] Peter J Denning. 程序行为的工作集模型。ACM通讯, 11(5): 323–333, 1968。
- [22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT：面向语言理解的深度双向Transformer预训练。2019。
- [23] 辛东, 陈尚宇, 潘新诺. 学习通过逐层最优脑外科手术方法剪枝深度神经网络。arXiv预印本 arXiv:1705.07565, 2017。
- [24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, 等. 一张图像抵得上16x16个单词：大规模图像识别的Transformer。载于国际学习表征会议, 2020。
- [25] Y Eidelman 和 I Gohberg. 关于一类新的结构化矩阵。积分方程与算子理论, 34(3):293–324, 1999。
- [26] Jean Feydy, Joan Glaunès, Benjamin Charlier 和 Michael Bronstein. 利用符号矩阵的快速几何学习。《神经信息处理系统进展》, 33, 2020年。
- [27] Jörg Flum 和 Martin Grohe. 参数化复杂性理论。Springer, 2006。[28] Jonathan Frankle 和 Michael Carbin. 彩票假说：寻找稀疏、可训练的神经网络。见国际学习表征会议, 2018。[29] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M Roy 和 Michael Carbin. 稳定彩票假说。arXiv 预印本 arXiv:1903.01611, 2019。
- [30] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, Michael Carbin. 线性模式连通性与彩票假说。载于国际机器学习大会论文集, 页码 3259–3269。PMLR, 2020。
- [31] Karan Goel, Albert Gu, Chris Donahue 和 Christopher Ré. 原始音频！基于状态空间模型的音频生成。见国际机器学习会议 (ICML), 2022。
- [32] Aaron Gokaslan, Vanya Cohen, Pavlick Ellie 和 Stefanie Tellex. Openwebtext 语料库, 2019。

[33] 吉姆·格雷, 苏拉吉特·乔杜里, 亚当·博斯沃思, 安德鲁·莱曼, 唐·赖卡特, 穆拉利·文卡特拉奥, 弗兰克·佩洛, 哈米德·皮拉赫什. 数据立方体：一种泛化分组、交叉表和子汇总的关系聚合算子. 数据挖掘与知识发现, 1(1):29–53, 1997. [34] 安德烈亚斯·格里万克, 安德烈娅·瓦尔特. 导数求值：算法微分的原理与技术. SIAM, 2008. [35] 阿尔伯特·古, 特里·道, 斯特凡诺·埃尔蒙, 阿特里·鲁德拉, 克里斯托弗·雷. Hippo：具有最优多项式投影的循环记忆. 收录于《神经信息处理系统进展》(NeurIPS), 2020. [36] 阿尔伯特·古, 艾西斯·约翰逊, 卡兰·戈埃尔, 哈立德·萨博, 特里·道, 阿特里·鲁德拉, 克里斯托弗·雷. 利用线性状态空间层结合循环、卷积和连续时间模型. 神经信息处理系统进展, 34, 2021.

[37] Albert Gu, Karan Goel 和 Christopher Ré. 利用结构化状态空间高效建模长序列. 发表于国际学习表征会议 (ICLR), 2022.

[38] 宋涵 (Song Han)、杰夫·普尔 (Jeff Pool)、约翰·特兰 (John Tran) 和威廉·J·戴利 (William J Dally)。《学习权重与连接以实现高效神经网络》。arXiv预印本 arXiv:1506.02626, 2015年。

[39] Song Han, Huizi Mao, 和 William J Dally. 深度压缩：通过剪枝、训练量化和霍夫曼编码压缩深度神经网络. 国际学习表征大会, 2016.

[40] John Hennessy 和 David Patterson. 存储器层次结构设计. 计算机体系结构：量化研究方法, 第390–525页, 2003.

[41] Sara Hooker. 硬件运气. arXiv预印本 arXiv:2009.06489, 2020.

[42] 华韦哲, 戴子航, 刘寒骁, Quoc V Le. 线性时间内的Transformer质量. arXiv预印本 arXiv:2202.10447, 2022.

[43] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, 和 Torsten Hoefler. 数据移动就是你所需的全部：优化Transformer的案例研究. 《机器学习与系统会议论文集》, 3: 711–732, 2021.

[44] Zhe Jia和Peter Van Sandt. 通过微基准测试剖析Ampere GPU架构. GPU技术大会, 2021.

[45] Zhe Jia, Marco Maggioni, Benjamin Staiger, 和 Daniele P Scarpazza. 通过微基准测试剖析英伟达 Volta GPU架构. arXiv 预印本 arXiv:1804.06826, 2018.

[46] Zhe Jia, Blake Tillman, Marco Maggioni, 与 Daniele Paolo Scarpazza. 通过微基准测试剖析Graphcore IPU架构. arXiv预印本 arXiv:1912.03413, 2019.

[47] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, 和 Roger G Mark. Mimic-iii, 一个可免费访问的重症监护数据库. 《科学数据》, 3(1):1–9, 2016年. [48] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, 等. 张量处理单元的数据中心内性能分析. 收录于《第44届计算机体系结构年度国际研讨会论文集》, 第1–12页, 2017年.

[49] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. 矩阵和线性方程的位移秩. Journal of Mathematical Analysis and Applications, 68(2):395–407, 1979.

[50] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers 是 RNN: 具有线性注意力的快速自回归 Transformer. 收录于国际机器学习大会论文集, 页码 5156–5165. PMLR, 2020.

- [51] Nikita Kitaev, Łukasz Kaiser 和 Anselm Levskaya. Reformer：高效的Transformer. 收录于《国际机器学习大会 (ICML)》，2020.
- [52] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, 和 Radu Soricut. Albert: 一种用于语言表征自监督学习的轻量级BERT. 在国际学习表征会议(ICLR), 2020.
- [53] 李明臻, 刘毅, 刘晓艳, 孙清霄, 尤昕, 杨海龙, 栾钟治, 甘霖, 杨广文, 钱德沛. 深度学习编译器：综述. 《IEEE 并行与分布式系统汇刊》，32(3):708–727, 2020.
- [54] Valerii Likhoshesterov, Krzysztof Choromanski, Jared Davis, Xingyou Song, and Adrian Weller. 亚线性记忆：如何让Performer模型更轻量. arXiv预印本 arXiv:2012.11346, 2020.
- [55] Ji Lin, Yongming Rao, Jiwen Lu, Jie Zhou. 运行时神经网络剪枝. 见 I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett 编, 《神经信息处理系统进展》，第30卷. Curran Associates, Inc., 2017.
- [56] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: 一种稳健优化的BERT预训练方法. arXiv preprint arXiv:1907.11692, 2019.
- [57] Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma 和 Luke Zettlemoyer. Luna: 线性统一嵌套注意力. 神经信息处理系统进展, 34, 2021.
- [58] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, 等人. MLPerf训练基准. 机器学习与系统会议论文集, 2:336–349, 2020.
- [59] Frank McSherry, Michael Isard 和 Derek G Murray. 可扩展性！但以怎样的{成本}为代价？载于第15届操作系统热点研讨会（HotOS XV），2015年。
- [60] Maxim Milakov 和 Natalia Gimelshein. Softmax的在线归一化因子计算. arXiv preprint arXiv:1805.02867, 2018.
- [61] NVIDIA. NVIDIA Tesla V100 GPU 架构，2017。
- [62] NVIDIA. Nvidia A100 张量核心 GPU 架构，2020。
- [63] NVIDIA. NVIDIA H100 Tensor Core GPU架构，2022。
- [64] D Stott Parker. 随机蝴蝶变换及其在计算线性代数中的应用. 1995.
- [65] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, 等. PyTorch：一种命令式风格的高性能深度学习库. 神经信息处理系统进展, 32, 2019.
- [66] Markus N Rabe 和 Charles Staats. 自注意力不需要 $O(n^2)$ 内存. arXiv preprint arXiv:2112.05682, 2021.
- [67] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever 等. 语言模型是无监督的多任务学习者. OpenAI 博客, 1(8):9, 2019.
- [68] Jack Rae 和 Ali Razavi. Transformer需要深度长程记忆吗？见：第58届计算语言学协会年会会议记录，线上，2020年7月。计算语言学协会。URL <https://www.aclweb.org/anthology/2020.acl-main.672>.
- [69] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, 和 Timothy P Lillicrap. 用于长程序列建模的压缩变压器. 在国际学习表征会议 (ICLR), 2020.

[70] Jonathan Ragan-Kelley、Connelly Barnes、Andrew Adams、Sylvain Paris、Frédéric Durand和Saman Amarasinghe. Halide：一种用于优化图像处理流水线中并行性、局部性和重计算的语言和编译器。《ACM Sigplan Notices》，48(6):519–530，2013年。[71] Raghu Ramakrishnan、Johannes Gehrke和Johannes Gehrke。《数据库管理系统》，第3卷。McGraw-Hill纽约，2003年。[72] Benjamin Recht和Christopher Ré。用于大规模矩阵补全的并行随机梯度算法。《数学规划计算》，5(2):201–226，2013年。[73] Hongyu Ren、Hanjun Dai、Zihang Dai、Mengjiao Yang、Jure Leskovec、Dale Schuurmans和Bo Dai。Combiner：具有稀疏计算成本的全注意力Transformer。《神经信息处理系统进展》，34，2021年。

[74] Aurko Roy, Mohammad Saffar, Ashish Vaswani, David Grangier. 基于内容的高效稀疏注意力与路由变换器. 《计算语言学协会汇刊》，9: 53–68, 2021.

[75] Amit Sabne. XLA：为峰值性能编译机器学习。2020.

[76] Victor Sanh、Thomas Wolf 和 Alexander M Rush. 移动剪枝：通过微调实现自适应稀疏性. arXiv 预印本 arXiv:2005.07683, 2020.

[77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, 与 Bryan Catanzaro. Megatron-LM：利用模型并行训练数十亿参数量级语言模型. arXiv预印本 arXiv:1909.08053, 2019.

[78] Vikas Sindhwani、Tara Sainath 和 Sanjiv Kumar. 面向小尺寸深度学习的结构化变换. 载于《神经信息处理系统进展》，第3088–3096页, 2015.

[79] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, 和 Armand Joulin. Transformer中的自适应注意力跨度. 载于计算语言学协会年会论文集, 2019.

[80] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, Donald Metzler. 长程竞技场：高效Transformer基准测试. 国际学习表征会议, 2020.

[81] 易泰, 穆斯塔法·德赫加尼, 达拉·巴赫里, 唐纳德·梅茨勒. 高效Transformer：一项综述. arXiv预印本 arXiv:2009.06732, 2020.

[82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, 与 Illia Polosukhin. 注意力即你所需. 神经信息处理系统进展, 30, 2017.

[83] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: 将Transformer扩展至千层规模. arXiv预印本 arXiv:2203.00555, 2022.

[84] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang 和 Hao Ma. Linformer：具有线性复杂度的自注意力机制. arXiv 预印本 arXiv:2006.04768, 2020. [85] Samuel Williams, Andrew Waterman 和 David Patterson. Roofline：一个面向多核架构的深刻可视化性能模型. 《ACM通讯》，52(4):65–76, 2009. [86] Michael E Wolf 和 Monica S Lam. 一种数据局部性优化算法. 收录于《1991年ACM SIGPLAN编程语言设计与实现会议论文集》，第30–44页, 1991.

- [87] 托马斯·沃尔夫, 利桑德·德布, 维克多·桑, 朱利安·肖蒙, 克莱门特·德朗格, 安东尼·莫伊, 皮埃里克·西斯塔克, 蒂姆·劳特, 雷米·洛夫, 摩根·丰托维奇, 乔·戴维森, 萨姆·施莱弗, 帕特里克·冯·普拉滕, 克拉拉·马, 雅辛·杰尼特, 朱利安·普卢, 徐灿文, 泰文·勒·斯卡奥, 西尔万·古格, 玛丽亚玛·德拉梅, 昆廷·洛斯特, 以及亚历山大·M拉什。Transformers: 最先进的自然语言处理。收录于《2020年自然语言处理经验方法会议: 系统演示论文集》, 第38-45页, 线上会议, 2020年10月。计算语言学协会。网址: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>。
- [88] David P Woodruff. 所有频率矩的最优空间下界. 见 SODA, 第4卷, 第167–175页. Citeseer, 2004.
- [89] 吴菲利克斯、范安吉拉、巴耶夫斯基阿列克谢、多芬扬恩·N和奥利迈克尔。通过轻量级与动态卷积实现更少注意力。载于《学习表征国际会议》(ICLR), 2019年。
- [90] 熊云阳, 曾占鹏, Rudrasis Chakraborty, 谭明兴, Glenn Fung, 李寅, Vikas Singh. Nyströmformer: 一种基于Nyström方法的自注意力近似算法。收录于《AAAI人工智能会议论文集》。AAAI人工智能会议, 第35卷, 第14138页, 2021年。
- [91] 李源, 陈云鹏, 王涛, 于伟豪, 石宇军, 蒋子航, Francis EH Tay, 冯佳时, 严水成. Tokens-to-token vit: 在Image Net上从头开始训练视觉transformer. 收录于IEEE/CVF国际计算机视觉会议论文集, 页码558–567, 2021.
- [92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, 等. Big bird: 面向更长序列的变换器. 《神经信息处理系统进展》, 33, 2020.
- [93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, 和 Josh Susskind. 一种无注意力机制的Transformer模型. arXiv预印本 arXiv:2105.14103, 2021.
- [94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, 和 Bryan Catanzaro. 长短Transformer: 高效的语言与视觉Transformer. 神经信息处理系统进展, 34, 2021.

相关工作

IO感知的运行时优化。针对快速/慢速存储器读写进行优化的广义概念，在计算机科学领域源远流长，并以多种名称广为人知。在本文中，我们最直接地关联到分析IO复杂度的相关文献[1]，但内存层级结构的概念是基础性的，并以多种形式出现，从工作集模型[21]、数据局部性[86]、算术强度的Roofline模型[85]、可扩展性分析[59]，到计算机体系结构的标准教科书论述[40]。我们希望这项工作能鼓励社区在深度学习技术栈的更多环节采纳这些理念。

基于结构化矩阵的高效机器学习模型。矩阵乘法是大多数机器学习模型的核心计算瓶颈。为了降低计算复杂度，出现了许多在更高效矩阵集合上学习的方法。这些矩阵被称为结构化矩阵，它们具有次二次方的（对于维度为 $n \times n$ 的矩阵为 $O(n^2)$ ）参数数量和运行时间。最常见的结构化矩阵例子包括稀疏矩阵和低秩矩阵，以及信号处理中常见的快速变换（傅里叶、切比雪夫、正弦/余弦、正交多项式）。机器学习中提出了几种更通用的结构化矩阵类别：类托普利茨矩阵 [78]、低位错秩矩阵 [49]、准可分离矩阵 [25]。我们用于块稀疏注意力的蝶形模式，其动机在于蝶形矩阵 [15, 64] 及其乘积已被证明能够以近乎最优的运行时间和参数数量表达任何结构化矩阵 [16, 20]。然而，尽管理论上结构化矩阵很高效，但由于密集无约束矩阵乘法具有高度优化的实现，因此很难将其效率转化为实际运行时的加速，这一现象被称为硬件彩票 [41]。蝶形矩阵的扩展 [17, 18] 旨在使其更硬件友好。

稀疏训练。我们的分块稀疏FlashAttention可视为迈向更高效稀疏模型训练的一步。稀疏模型已通过对权重矩阵进行稀疏化处理，在推理（剪枝）压缩模型方面取得了成功[23, 38, 39, 55, 76]。对于模型训练，彩票假说[28, 29, 30]表明，存在一组源自更大密集网络的子网络，其性能可与原始密集网络相媲美。在注意力机制的背景下，我们的分块稀疏FlashAttention也可被视为一种固定彩票：我们在训练过程中将稀疏模式固定为蝴蝶模式，并观察到在长程竞技场任务上，其性能几乎与（密集）FlashAttention相当。

高效Transformer。基于Transformer的模型已成为自然语言处理[22]和计算机视觉[24, 91]中应用最广泛的架构。然而，其计算瓶颈之一是时间和内存随序列长度呈二次方增长。针对这一瓶颈，存在众多方法，包括基于哈希（即稀疏）的近似，如Reformer[51]和Smyrf[19]，以及基于低秩近似的Performer[12, 54]。我们甚至可以将稀疏和低秩近似结合以获得更好的精度（例如Longformer[3]、BigBird[92]、Scatterbrain[9]、长短Transformer[94]、Combiner[73]）。其他方法包括沿序列维度进行压缩，以便一次关注多个token[52, 57, 79, 89]。还可以关注来自先前序列的状态以帮助延长上下文（例如Transformer-XL[14]和Compressive Transformer[69]）。我们推荐查阅综述[81]以获取更多细节。

为建模更长的上下文，探索注意力机制替代模块的研究有多个方向。HiPPO[35]及其扩展（尤其是S4[31, 36, 37]）在多项式基上对历史信息进行投影，能够通过状态空间模型精确重建历史。这些方法融合了CNN（高效训练）、RNN（高效推理）和连续模型（对采样率变化鲁棒）的优势。LambdaNetworks[2]、AFT[93]和FLASH[42]则是在图像分类和语言建模中替换注意力机制的其他尝试。

B 算法详情

我们首先推导注意力的前向传播与反向传播，并证明它们可以以内存高效的方式计算（所需的额外内存与序列长度呈线性关系，而非二次关系）。尽管它们减少了所需的额外内存量，但原始实现仍会导致 HBM 的二次访问，从而降低执行速度。我们描述了 FlashAttention 算法，以实现前向传播

以及 GPU 上的反向传播减少了 HBM 访问，从而实现了更快的运行速度和更小的内存占用。

B.1 内存高效的前向传播

使注意力机制内存高效的主要挑战在于 softmax 将 $\mathbf{K}$ (的列与 $\mathbf{V}$) 的列耦合在一起。我们的方法是单独计算 softmax 归一化常数以解耦这些列。文献 [51, 66] 中使用了这一技术 [60]，以表明注意力计算不需要二次方的额外内存（尽管 HBM 访问次数仍为二次方，导致运行速度缓慢）。

为简化起见，我们在此省略了 softmax 过程中的最大值平移步骤。附录 B.3 中的完整算法包含了所有步骤。

回想一下，给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，我们要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

我们有 $i_j = \frac{1}{L_i} \sum_j$ ，其中 i 和 j 分别是 $\mathbf{Q}$ 和 $\mathbf{K}$ 的第 i 列和第 j 列。定义 softmax 的归一化常数：

$$L_i = \sum_j e^{q_i^T k_j}. \quad (1)$$

设 j 为 $\mathbf{V}$ 的第 j 列，则输出的第 i 列为

$$o_i = P_{i:} \mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \quad (2)$$

我们看到，一旦计算出 L_i ，就可以通过反复求和 $\frac{e^{q_i^T k_j}}{L_i} v_j$ 来计算 o_i ，无需额外内存。因此，前向传播可以用 (1) 的额外内存计算：

根据公式(1)对所有 i 计算 L_i ，这需要 (1) 的额外内存。

2. 根据公式(2)计算所有 o_i 的，这需要 (2) 的额外内存。

B.2 内存高效的反向传播

我们推导了注意力机制的反向传播过程，并证明其同样可在线性内存下完成计算。Rabe 与 Staats [66] 提出，通过将梯度检查点应用于内存高效的前向传播，反向传播可以避免使用平方级的额外内存。而我们则显式地推导了反向传播过程，并展示了如何以内存高效的方式实现该计算。

假设存在一个标量损失函数 ϕ ，记输出梯度为 $d\mathbf{O} \in \mathbb{R}^{n \times d}$ (其中 $d\mathbf{O}$ 表示 $\frac{\partial \phi}{\partial \mathbf{O}}$)。我们需要计算输入梯度 $d\mathbf{Q}$ 、 $d\mathbf{K}$ 、 $d\mathbf{V} \in \mathbb{R}^{n \times d}$ (其中 $d\mathbf{Q}$ 、 $d\mathbf{K}$ 、 $d\mathbf{V}$ 分别记作 $\frac{\partial \phi}{\partial \mathbf{Q}}$ 、 $\frac{\partial \phi}{\partial \mathbf{K}}$ 、 $\frac{\partial \phi}{\partial \mathbf{V}}$)。

梯度 $d\mathbf{V}$ 很容易看出。手动应用反向模式自动微分（即链式法则），我们得到（用矩阵表示法） $d\mathbf{V} = \mathbf{P}^T d\mathbf{O}$ 。因此：

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \quad (3)$$

由于我们已经计算了 L_i ，因此通过重复求和即可计算 dv_j ，无需额外内存。

梯度 $d\mathbf{Q}$ 和 $d\mathbf{K}$ 稍微复杂一些。我们先来看梯度 $d\mathbf{P}$ 和 $d\mathbf{S}$ 。根据公式(2)，我们有 $d\mathbf{P} = d\mathbf{O}\mathbf{V}^T$ ，因此：

$$dP_{ij} = do_i^T v_j.$$

回想一下， $P_{i:} = \text{softmax}(S_{i:})$ 。利用以下事实： $P_{i:} = \text{softmax}(S_{i:})$ 的雅可比矩阵是 $\text{diag}(P_{i:}) - P_{i:}^T$ ，我们可以得到

$$dS_{i:} = (\text{diag}(P_{i:}) - P_{i:} P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

其中 $\circ$ 表示逐元素乘法。

定义

$$D_i = P_i^T dP_i = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j = do_i^T o_i, \quad (4)$$

然后

$$dS_i = P_i \circ dP_i - D_i P_i.$$

因此

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij} (dP_{ij} - D_i).$$

现在 w 我们可以得到梯度 dQ 和 dK 。回想一下， $ij = \frac{T}{i}$ ，因此

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij} (dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) k_j. \quad (5)$$

同样地，

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij} (dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} (do_i^T v_j - D_i) q_i. \quad (6)$$

因此，反向传播也可以使用 () 的额外内存来计算：

1. 根据公式(3)为所有 j 计算 k_j ，这需要 () 的额外内存。
2. 根据式(4)对所有 i 计算 D_i ，这需要 () 额外内存。
3. 根据公式(5)对所有 i 计算 dq_i ，这需要 () 的额外内存。
4. 根据公式(6)计算所有 j 的 dk_j ，这需要额外的 () 内存。

B.3 FlashAttention：前向传播

我们描述FlashAttention前向传递的完整细节。给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，我们希望计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \tau \mathbf{Q} \mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$

$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 $\tau \in \mathbb{R}$ 是某种 softmax 缩放（通常为 $\frac{1}{\sqrt{d}}$ ），mask 是某种掩码函数，将输入的一些条目设置为 $-\infty$ 而保持其他条目不变（例如，当批次中的序列长度不同而进行了填充时使用的键填充掩码）， $\text{dropout}(\cdot, p)$ 对 逐元素应用 dropout（即，对于每个元素 x ，以概率 p 输出 $\frac{x}{1-p}$ ，以概率 $1-p$ 输出 0）。

完整算法见算法2。我们保存输出 $\mathbf{O}$ 、softmax统计量 ℓ 和 $\mathbf{P}$ ，以及伪随机数生成器状态 $\mathbf{R}$ ，以供反向传播使用。

算法 2 FlashAttention 前向传播 要求HBM中的矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, 大小为 的片上 SRAM, softmax 缩放常数 $\beta \in \mathbb{R}$, 掩码函数 mask , 丢弃概率 dropo 1: 初始化伪随机数生成器状态 $\mathcal{R}$ 并保存至 HBM。 2: 设置分块大小 $c = \lceil \frac{M}{4d} \rceil$, $r = \min(\lceil \frac{M}{4d} \rceil, \dots)$ 。 3: 在 HBM 中初始化 $\mathbf{O} = (\leftarrow 0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell \leftarrow = (0)_N \in \mathbb{R}^N$, $\leftarrow = (-\infty)_N \in \mathbb{R}^N$ 。 4: 将 $\mathbf{Q}$ 按大小 $r \times \times$ 分为 $r = \lceil \frac{N}{B_r} \rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$, 并将 $\mathbf{K}, \mathbf{V}$ 按大小 $c \times \times$ 分为 $c = \lceil \frac{N}{B_c} \rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 和 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ 。 5: 将 $\mathbf{O}$ 按大小 $r \times \times$ 分为 r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$, 将 ℓ 按大小 r 分为 r 个块 $\ell_1, \dots, \ell_{T_r}$, 将 按大小 r 分为 r 个块 $\mathbf{1}, \dots, \mathbf{1}_{T_r}$ 。 6: for $1 \leq \leq c$ do 7: 从 HBM 加载 $\mathbf{K}_j, \mathbf{V}_j$ 到片上 SRAM。 8: for $1 \leq \leq r$ do 9: 从 HBM 加载 $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, \mathbf{1}_i$ 到片上 SRAM。 10: 在片上计算 $\mathbf{S}_{ij} = \leftarrow \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。 11: 在片上计算 $\mathbf{S}_{ij}^{\text{masked}} = \leftarrow \text{mask}(\mathbf{S}_{ij})$ 。 12: 在片上计算 $\leftarrow_{ij} = \leftarrow \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \leftarrow \exp(\mathbf{S}_{ij}^{\text{masked}} - \leftarrow_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\leftarrow_{ij} = \leftarrow \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$ 。 13: 在片上计算 $\mathbf{1}_i^{\text{new}} = \leftarrow \max(\mathbf{1}_i, \leftarrow_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = \leftarrow m_i - m_i^{\text{new}} \ell_i + - \tilde{m}_{ij} - m_i^{\text{new}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$ 。 14: 在片上计算 $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \leftarrow \text{dropout}(\tilde{\mathbf{P}}_{ij}, \text{dropo})$ 。 15: 将 $\mathbf{O}_i \leftarrow \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (-\text{diag}(\ell_i) m_i - m_i^{\text{new}} \mathbf{O}_i + - \tilde{m}_{ij} - m_i^{\text{new}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ 写入 HBM。 16: 将 $\ell_i \leftarrow \leftarrow \ell_i^{\text{new}}$, $\mathbf{1}_i \leftarrow \leftarrow \mathbf{1}_i^{\text{new}}$ 写入 HBM。 17: end for 18: end for 19: 返回 $\mathbf{O}, \ell, \mathbf{1}, \mathcal{R}$ 。

B.4 FlashAttention：反向传播

我们详细描述FlashAttention反向传播的全部细节。给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, 输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$, 以及输出梯度 $d\mathbf{O}$, 我们希望计算输入梯度 $d\mathbf{Q}, d\mathbf{K}$ 和 $d\mathbf{V} \in \mathbb{R}^{N \times d}$ 。为了完整性, 我们首先在算法3中描述标准的注意力反向传播。

算法3 标准注意力反向传播

要求：矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, d\mathbf{O} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ 在 HBM 中。

- 1: 从HBM以块形式加载 $\mathbf{P}, d\mathbf{O}$, 计算 $d\mathbf{V} = \mathbf{P}^T d\mathbf{O} \in \mathbb{R}^{N \times d}$, 将 $d\mathbf{V}$ 写入 HBM。 2: 从HBM以块形式加载 $d\mathbf{O}, \mathbf{V}$, 计算 $d\mathbf{P} = d\mathbf{O} \mathbf{V}^T \in \mathbb{R}^{N \times N}$, 将 $d\mathbf{P}$ 写入 HBM。 3: 从HBM读取 $\mathbf{P}, d\mathbf{P}$, 计算 $d\mathbf{S} \in \mathbb{R}^{N \times N}$, 其中 $ij = ij(\mathbf{1}_i - \sum_l \mathbf{1}_l \mathbf{1}_l^T)$, 将 $d\mathbf{S}$ 写入 HBM。 4: 从HBM以块形式加载 $d\mathbf{S}$ 和 $\mathbf{K}$, 计算 $d\mathbf{Q} = d\mathbf{S} \mathbf{K}$, 将 $d\mathbf{Q}$ 写入 HBM。 5: 从HBM以块形式加载 $d\mathbf{S}$ 和 $\mathbf{Q}$, 计算 $d\mathbf{K} = d\mathbf{S}^T \mathbf{Q}$, 将 $d\mathbf{K}$ 写入 HBM。 6: 返回 $d\mathbf{Q}, d\mathbf{K}, d\mathbf{V}$ 。
-

我们现在对 FlashAttention 反向传播做两个观察：

1. 我们不需要存储来自前向传播的大小为 $(\frac{N}{B_r})^2$ 的 dropout 掩码。相反, 我们可以保存前向传播中的伪随机数生成器状态, 并在反向传播中重新生成 dropout 掩码。这使我们只需使用 $(\frac{N}{B_r})$ 的额外内存。
2. 当计算 softmax 梯度时, 我们使用式(4)来计算 $\mathbf{1}_i = \frac{\mathbf{1}_i}{\sum_l \mathbf{1}_l}$, 而不对大小为 $\frac{N}{B_r}$ 的 $\mathbf{1}_i$ 和 $\mathbf{1}_l$ 进行归约 (它们可能不适合 SRAM)。相反, 我们可以重写 $\mathbf{1}_i = \frac{\mathbf{1}_i}{\sum_l \mathbf{1}_l}$ 并计算大小为 $\frac{N}{B_r}$ 的向量之间的点积。

完整的 FlashAttention 反向传播算法如算法4所示。从概念上讲，它只是附录B.2中推导的块形式。

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , pseudo-random number generator state $\mathcal{R}$ from the forward pass.

- 1: Set the pseudo-random number generator state to $\mathcal{R}$.
- 2: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}$, $\mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ into T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
- 6: **for** $1 \leq j \leq T_c$ **do**
- 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 8: Initialize $\tilde{\mathbf{dK}}_j = (0)_{B_c \times d}$, $\tilde{\mathbf{dV}}_j = (0)_{B_c \times d}$ on SRAM.
- 9: **for** $1 \leq i \leq T_r$ **do**
- 10: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 11: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 12: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
- 13: On chip, compute $\mathbf{P}_{ij} = \text{diag}(\ell_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
- 14: On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop} .
- 15: On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
- 16: On chip, compute $\tilde{\mathbf{dV}}_j \leftarrow \tilde{\mathbf{dV}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
- 17: On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 18: On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
- 19: On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
- 20: On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 21: Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
- 22: On chip, compute $\tilde{\mathbf{dK}}_j \leftarrow \tilde{\mathbf{dK}}_j + \tau \mathbf{dS}_{ij}^T \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
- 23: **end for**
- 24: Write $\mathbf{dK}_j \leftarrow \tilde{\mathbf{dK}}_j$, $\mathbf{dV}_j \leftarrow \tilde{\mathbf{dV}}_j$ to HBM.
- 25: **end for**
- 26: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

我们看到，与正向传播类似，反向传播执行 $(\frac{N}{B_r})^2$ 次浮点运算，并且除了输入、输出、输出梯度和输入梯度之外，仅需要 $(\frac{N}{B_r})^2$ 的额外内存。

我们分析反向传递的IO复杂度，类似于正向传递（定理2）。

定理5. 设 N 为序列长度， d 为头维度，且 M 为RAM的大小，满足 $\frac{N}{B_r} \leq \frac{M}{4d} \leq \frac{N}{B_c}$ 。标准注意力（算法）的反向传播需要 $\Theta(\frac{N^2}{B_r} + \frac{N^2}{B_c})$ 次HBM访问，而FlashAttention的反向传播（算法4）需要 $\Theta(\frac{N^2}{B_r} + \frac{N^2}{B_c} - 1)$ 次HBM访问。

证明见附录C。

B.5 与Rabe和Staats [66]的比较

我们在此描述我们的FlashAttention算法与Rabe和Staats [66]算法之间的一些相似之处和差异。

从概念上讲，FlashAttention 与 Rabe 和 Staats [66] 的方法均采用了成熟的分块（或 softmax 缩放）技术 [51, 60] 在注意力矩阵的块上进行操作。为减少内存占用，这两种方法都避免了在前向传播中存储大型注意力矩阵，而是在反向传播时重新计算它。

第一个主要区别在于，Rabe和Staats [66]侧重于减少总内存占用（所需的最大GPU内存量），而FlashAttention则侧重于减少内存访问次数（内存读写次数）。如第2节所述，内存访问量是运行时的主要决定因素。减少内存访问也必然会减少所需的总内存量（例如，如果一个操作产生 c 次内存访问，那么其总内存需求最多为 $c \times B_r \times B_c \times d$ ）。因此，FlashAttention比标准注意力机制更快（2-4 $\times$ ），而Rabe和Staats [66]的速度与标准注意力机制大致相同或略慢。就所需总内存而言，两种方法都能大幅节省内存。

这两种方法的第二个区别在于每个块向下一块传递信息时的摘要方式。Rabe和Staats[66]通过临时输出以及softmax归一化统计量来总结每个块。在前向传播结束时，所有块的临时输出利用这些统计量组合起来以产生最终输出。而FlashAttention则在处理每个块后增量式地更新输出（算法1第12行），因此只需保留一份输出（而不是为K个块保留K份副本）。这意味着与Rabe和Staats[66]的方法相比，FlashAttention的总内存需求更小。

最后一个主要区别在于反向传播的计算方式。Rabe 和 Staats [66] 使用梯度检查点技术来重新计算注意力矩阵和每个块的临时输出。FlashAttention 则通过解析方式简化了反向传播（附录 B.2 和 B.4）。它仅重新计算注意力矩阵，而不重新计算每个块的临时输出。这降低了反向传播的内存需求，并带来了速度提升。

C 证明

定理1的证明。我们首先计算所需的FLOPs数量和额外内存。

主要计算量来自矩阵乘法。在内层循环中（算法1第9行），我们为 $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ 和 $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$ 计算 $\mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ ，这需要 $(r \times c)$ 次浮点运算。同时（算法1第12行）为 $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$ 和 $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$ 计算 $\tilde{\mathbf{P}}_{ij} \mathbf{V}_j \in \mathbb{R}^{B_r \times d}$ ，需要 $(r \times c)$ 次浮点运算。我们执行内层循环共 $c \times r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$ 次，因此总浮点运算次数为

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d).$$

就所需的额外内存而言，我们看到需要 $(r \times c)$ 的内存来存储统计量 $(\ell, \mathbf{O})$ 。

我们现在通过对 j 进行归纳来证明算法的正确性，其中 $0 \leq j \leq c_0$ 。令 $\mathbf{K}_{:,j} \in \mathbb{R}^{j B_c \times d}$ 为 $\mathbf{K}$ 的前 $j B_c$ 行，类似地， $\mathbf{V}_{:,j} \in \mathbb{R}^{j B_c \times d}$ 为 $\mathbf{V}$ 的前 $j B_c$ 行。令 $\mathbf{S}_{:,j} = \mathbf{Q} \mathbf{K}_{:,j}^\top \in \mathbb{R}^{N \times j B_c}$ ，及 $\mathbf{P}_{:,j} = \text{softmax}(\mathbf{S}_{:,j}) \in \mathbb{R}^{N \times j B_c}$ （按行应用的 softmax）。令 $\mathbf{m}^{(j)}$ 、 $\ell^{(j)}$ 、 $\mathbf{O}^{(j)}$ 为外层循环（算法1第5行）第 j 次迭代后 HBM 中 $\mathbf{m}$ 、 ℓ 、 $\mathbf{O}$ 的值。（注意，这些 $\mathbf{m}$ 、 ℓ 、 $\mathbf{O}$ 的值在外层循环的每次迭代后都会更新。）我们想要证明，在外层循环的第 j 次迭代后，我们已在 HBM 中计算得到：

$$\mathbf{m}^{(j)} = \text{rowmax}(\mathbf{S}_{:,j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \text{rowsum}(\exp(\mathbf{S}_{:,j} - \mathbf{m}^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,j} \mathbf{V}_{:,j} \in \mathbb{R}^{N \times d}.$$

基于我们的初始化（算法 1 第 2 行），该论断对于 $j=0$ 成立（即在外层循环的任何迭代执行之前）。假设该论断对于某个 $j=0, \dots, c_0-1$ 成立。我们想要证明该论断对于 $j+1$ 也成立。实际上，当我们在内循环中更新统计数据时（算法 1 第 10 行）

在外部循环的第 $(j+1)$ 迭代中，我们更新 $m^{(j+1)} = \max(\ell^{(j)}, \tilde{m})$ ，其中 $\tilde{m} \in \mathbb{R}^N$ 是 $S_{:,j:j+1}$ 的行最大值，即 S 从第 c 列到第 $(c+1)$ 列的切片。这意味着

$$m^{(j+1)} = \text{rowmax}(S_{:,j:j+1}) \in \mathbb{R}^N.$$

同样地，我们更新

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

其中 $\tilde{\ell} = \text{rowsum}(\exp(S_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$ 。通过第3.1节中相同的代数操作，我们得到：

$$\ell^{(j+1)} = \text{rowsum}(\exp(S_{:,j:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

设 $V_{j:j+1}$ 为 V 从列 c 到列 $(c+1)$ 的切片，我们也更新：

$$\begin{aligned} \mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1} (\text{diag}(\ell^{(j)}) e^{m^{(j)} - m^{(j+1)}} \mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(S_{j:j+1} - \tilde{m}) V_{j:j+1}) \\ &= \text{diag}(\ell^{(j+1)})^{-1} (\text{diag}(\ell^{(j)}) e^{m^{(j)} - m^{(j+1)}} \mathbf{P}_{:,j} V_{:,j} + e^{-m^{(j+1)}} \exp(S_{j:j+1}) V_{j:j+1}) \\ &= \text{diag}(\ell^{(j+1)})^{-1} (\text{diag}(\ell^{(j)}) e^{m^{(j)} - m^{(j+1)}} \text{diag}(\ell^{(j)}) \exp(S_{:,j} - m^{(j)}) V_{:,j} + e^{-m^{(j+1)}} \exp(S_{j:j+1}) V_{j:j+1}) \\ &= \text{diag}(\ell^{(j+1)})^{-1} (e^{-m^{(j+1)}} \exp(S_{:,j}) V_{:,j} + e^{-m^{(j+1)}} \exp(S_{j:j+1}) V_{j:j+1}) \\ &= \text{diag}(\ell^{(j+1)})^{-1} (\exp(S_{:,j} - m^{(j+1)}) V_{:,j} + \exp(S_{j:j+1} - m^{(j+1)}) V_{j:j+1}) \\ &= \text{diag}(\ell^{(j+1)})^{-1} \left(\exp \left(\begin{bmatrix} S_{:,j} & S_{j:j+1} \end{bmatrix} - m^{(j+1)} \right) \right) \begin{bmatrix} V_{:,j} \\ V_{j:j+1} \end{bmatrix} \\ &= \text{softmax}(S_{:,j+1}) V_{:,j+1}. \end{aligned}$$

由此我们看到该断言对于 $c+1$ 也成立。根据归纳法，该断言对所有 $c = 0, \dots, c$ 成立。

当 $c = c$ 时，我们得出结论：HBM 中 $\mathbf{O}$ 的最终值为 $\text{softmax}(\mathbf{S})\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ 。

□

定理 2 的证明。我们首先分析标准注意力实现的 IO 复杂度。输入 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 驻留在 HBM 中，并且在算法结束时，输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 被写入 HBM。在计算矩阵乘法 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$ 的第一步中，从 HBM 读取输入 $\mathbf{Q}, \mathbf{K}$ ，并将输出 $\mathbf{S} \in \mathbb{R}^{N \times N}$ 写入 HBM（算法 0 第 1 行）。这导致 $\Theta(c + 2)$ 次 HBM 访问。

在计算 $\mathbf{P} = \text{softmax}(\mathbf{S})$ 的第二步中，输入 $\mathbf{S}$ 从 HBM 读取，输出 $\mathbf{P}$ 写入 HBM（算法 0 第 2 行）。这导致 $\Theta(c + 2)$ 次 HBM 访问。

在计算 $\mathbf{O} = \mathbf{P}\mathbf{V}$ 的最后一步中，从全局内存读取输入 $\mathbf{P}, \mathbf{V}$ ，并将输出 $\mathbf{O}$ 写入 HBM（算法 0 第 3 行）。这会导致 $\Theta(c + 2)$ 次 HBM 访问。

总体而言，标准注意力实现需要 $\Theta(c + 2)$ 次全局内存访问。

我们现在分析流式注意力的 IO 复杂度。

根据算法 1，我们看到 $\mathbf{K}$ 和 $\mathbf{V}$ 的每个元素从 HBM 加载一次（算法 1 第 6 行）。我们对 $\mathbf{Q}$ 和 $\mathbf{O}$ 进行 c 次遍历，每次遍历将所有 $\mathbf{Q}$ 和所有 $\mathbf{O}$ 加载到 HBM（算法 1 第 8 行）。因此，HBM 访问次数为 $\Theta(c + c) = \Theta(c)$ 。

我们推导了关于块大小 c 和 r 的条件。我们需要大小为 $c \times r$ 的块 $\mathbf{K}_j$ 和 $\mathbf{V}_j$ 适配片上存储，这等价于：

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

类似地，我们需要大小为 $r \times c$ 的块 $\mathbf{Q}_i, \mathbf{O}_i$ 装入片上内存，这意味着：

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

最后，我们需要大小为 $r \times c$ 的块 $\mathbf{S}_{ij}$ 能够装入片上内存，这意味着：

$$B_r B_c = O(M).$$

因此我们设定：

$$B_c = \Theta\left(\frac{M}{d}\right), \quad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

那么，我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM 访问次数为：

$$\Theta(NdT_c) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

命题3的证明。为了矛盾起见，假设存在一个计算精确注意力的算法，其中所有 $\in [,]$ 的HBM访问次数为

$$o\left(\frac{N^2d^2}{M}\right).$$

在寄存器中 $= \Theta()$ 的时间，这导致 HBM 访问次数 税收：

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

然而，注意力的输入（矩阵 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ ）和输出 $\mathbf{O}$ 的大小为 $N \times d$ ，并且它们最初存储在HBM中，因此如果该算法计算精确注意力，它必然会导致至少 $\Omega()$ 次 HBM 访问。这就产生了矛盾。□

定理5的证明。注意力反向的IO复杂度与注意力正向的IO复杂度非常相似（定理2）。这里我们提供证明的概要。

我们首先分析标准注意力反向传播的IO复杂度。输入 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 、 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ 驻留在HBM中，算法结束时输出 $\mathbf{dQ}$ 、 $\mathbf{dK}$ 、 $\mathbf{dV} \in \mathbb{R}^{N \times d}$ 被写入HBM。

在标准注意力反向传播的每一步中，需要从 HBM 加载大小为 $N \times d$ 或 $d \times d$ 的输入，并将大小为 $d \times d$ 或 $N \times d$ 的输出写入 HBM。这会导致 $\Theta()$ 次 HBM 访问。

我们现在分析FlashAttention反向传播的IO复杂度。

与定理2类似，我们看到 $\mathbf{K}$ 和 $\mathbf{V}$ 的每个元素都从HBM加载一次。 $\mathbf{dK}$ 和 $\mathbf{dV}$ 的每个元素仅被写入HBM一次。我们对 $\mathbf{Q}$ 、 $\mathbf{O}$ 、 $\mathbf{dO}$ 进行 c 趟遍历，每趟将所有的 $\mathbf{Q}$ 、 $\mathbf{O}$ 、 $\mathbf{dO}$ 加载到HBM中。我们还对 $\mathbf{dQ}$ 进行 c 趟遍历，每趟从HBM读取/写入所有的 $\mathbf{dQ}$ 。因此，HBM访问次数为 $\Theta() = \Theta()$ 。

正如在专业中定理2中，对块大小的约束为

在：

$$B_c = \Theta\left(\frac{M}{d}\right), \quad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

那么，我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM 访问次数为：

$$\Theta(NdT_c) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Algorithm 5 Block-Sparse FLASHATTENTION Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M , softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop} , block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$, block sparsity mask $M \in \{0, 1\}^{N/B_r \times N/B_c}$.

- 1: Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: **if** $M_{ij} \neq 0$ **then**
- 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 10: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 11: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
- 12: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 13: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 14: On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
- 15: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
- 16: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 17: **end if**
- 18: **end for**
- 19: **end for**
- 20: Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D 扩展详情

D.1 块稀疏 FlashAttention

我们在算法5中描述了完整的块稀疏FlashAttention算法。该算法与算法2相同，只是我们跳过了零块。

我们证明了块稀疏FlashAttention的IO复杂度。

命题4的证明。该证明与定理2的证明非常相似。针对块稀疏情形，需注意我们只需加载与非零块对应的块。因此，HBM访问次数会按块稀疏掩码中非零块的比例进行缩放。然而，当值较小时，我们仍需写入结果 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 。故HBM访问次数为

$$\Theta \left(Nd + \frac{N^2 d^2}{M} s \right).$$

□

D.2 潜在的扩展

我们在此讨论将IO感知方法扩展应用于加速深度学习训练的几种可能方式。

多GPU注意力机制。大型语言模型在数百或数千个GPU上进行训练，通常将同一节点上4-8个GPU之间的注意力计算进行拆分[77]。这引入了另一个层面的内存层次结构：除了GPU SRAM和GPU HBM之外，我们还需要考虑其他

GPU。对于非常长的序列，同一节点上的不同GPU可以通过考虑不同内存层级的不对称性来协作计算注意力。

稀疏 MLP 层。典型的密集 MLP 层是计算受限而非内存受限的。为了提高效率，可以使用具有稀疏权重矩阵的 MLP 层 [17]。然而，许多稀疏 MLP 层反而成为内存受限的，其加速往往与稀疏程度不成比例。我们相信，IO 感知的实现可以缓解这一问题，并实现稀疏性的优势。我们对这一方向的未来工作感到兴奋，以降低大型模型的计算需求并改善其实际运行时间。

核机器学习。我们在FlashAttention中的方法依赖于这样一个事实： $\times \times$ 的注意力矩阵是一个低秩矩阵 $\mathbf{QK}^T$ (的函数，该矩阵的秩为 $\ll \times$)。因此，我们可以重复加载输入 $\mathbf{Q}$ 、 $\mathbf{K}$ 并重新计算所需的注意力矩阵块，从而显著减少HBM访问。类似的情况也发生在核机器学习中： $\times \times$ 核矩阵 $\mathbf{K}$ 的每个元素 i, j 都是两个大小为 $\ll \times$ 的向量的函数，用以衡量两个数据点 i 与 j 之间的相似度。KeOps库[8, 26]是一个成功范例，展示了减少内存读写如何加速核运算。我们希望这将推动核方法更多地关注减少IO，而非仅仅专注于FLOPs的降低。

E 完整实验结果

E.1 BERT

我们参照 MLPerf 1.1 参考实现的训练过程和超参数来训练 BERT-large。具体而言，我们使用 LAMB 优化器，学习率设为 $3.75e-3$ ，批次大小为 448，最多训练 7100 步。一旦验证准确率（针对掩码语言模型）达到目标 72.0%，训练即停止，并测量实际运行时间。我们使用 Apex AMP（优化级别为 O2）以 FP16 精度进行训练。

我们将我们的结果与Nvidia提交给MLPerf 1.1（表1）的训练速度报告进行了比较。

我们采用 MLPerf 1.1 参考实现提供的相同训练/验证数据划分。具体而言，我们评估了与 Nvidia 基线相同的 10000 个验证样本。

我们在8xA100-80GB GPU上训练模型。每次训练运行需要16到19分钟，我们对10次运行的结果取平均值。

E.2 GPT-2

我们使用Huggingface transformers库和Nvidia的Megatron-LM仓库中的GPT-2 [67]标准实现。我们遵循Megatron-LM仓库的训练方案。

我们使用有效批量大小为512，并采用梯度累积以适应可用的GPU内存。我们使用AdamW优化器，GPT-2 small的学习率为 $6e-4$ ，GPT-2 medium为 $1.5e-4$ ，权重衰减为0.1。所有模型使用相同的超参数训练40万步。我们使用混合精度训练（PyTorch AMP）运行所有实现。

我们使用Openwebtext数据集，并采用GPT-2 BPE分词器。我们随机选取数据集的0.5%作为验证集，其余用作训练集。此验证集的随机选取仅执行一次，所有模型均在相同的验证集上进行评估。

我们在8x块A100-40GB GPU上训练模型，并测量实际训练时间。训练GPT-2小型模型需要2.7至9.5天，训练GPT-2中型模型需要6.9至21.0天（表2）。

在图4中，我们绘制了GPT-2 small/medium模型训练过程中的验证困惑度，分别使用了HuggingFace实现或我们的FlashAttention实现。可以看出，FlashAttention的表现与基准实现一致，两种实现的验证困惑度曲线几乎完全重叠。

长文档分类。对于MIMIC-III和ECtHR，我们遵循Dai等人[13]的超参数设置。

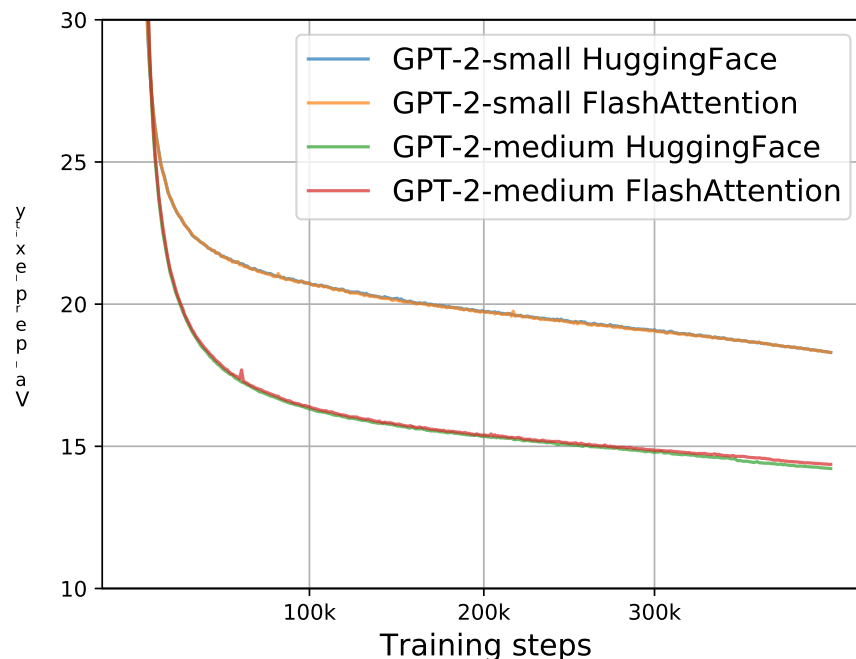


图4：使用两种实现方式的GPT-2 small/medium验证困惑度。我们确认FlashAttention能给出与HuggingFace基准实现相同的验证曲线。

E.3 LRA 详细信息

我们遵循长距离竞技场论文 [80]、长距离竞技场代码库 (<https://github.com/google-research/long-range-arena>) 以及 Nyströmformer 复现研究 [90] 中的超参数设置。为了对基线方法更为宽容，如果我们在五项任务中无法复现任一基线的性能，则对于该基线在该任务上的表现，我们将报告 Tay 等人 [80] 或 Xiong 等人 [90] 所给出的更优结果。

超参数调整后，几乎所有注意力机制方法均在全部五个LRA任务上达到了相似的准确率。

我们采用混合精度训练运行所有方法，但Performer（混合精度下表现不稳定）和Local Attention（其实现不支持FP16）除外。

为了计算整体墙钟时间加速比，我们取五个任务各自墙钟时间加速比的几何平均值。

Path-X 对于 Path-X 和 Path-256，我们沿用长序列基准论文[80]中 PathFinder-32 实验的超参数设置。针对这两类任务，我们首先在 Path-64 上预训练一个模型，取200个epoch后的检查点，对其位置嵌入进行上采样（通过空间网格复制位置嵌入），然后在下游任务上微调200个epoch，采用一个epoch的线性预热和余弦学习率衰减策略。对于 Path-X，我们选取表现最优的检查点（依据验证集准确率），再以相同的预热机制和学习率进行200个epoch的追加微调（这能为 Path-X 上的 FlashAttention 带来约4个百分点的准确率提升，但后续模型将开始过拟合）。

E.4 与 Apex 融合多头注意力的对比

我们将我们的方法/实现与Apex FMHA (<https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha>) 进行比较。

当我们启动这个项目时，Apex FMHA 是我们所知最快的注意力实现，专为长度不超过512的短序列量身定制。事实上，截至 MLPerf 1.1 [58]，几乎所有在 Nvidia GPU 上运行的 BERT 训练基准测试的 MLPerf 提交都使用 FMHA 作为其模型代码。由于

表 7：FlashAttention 与 FMHA 的运行时（毫秒）对比，按序列长度划分，包含掩码和 dropout，在 A100-SXM4-40GB GPU 上测量。批量大小 64，头数 16，头维度 64（即 BERT-large 规模）。

Attention Method	128	256	512
Apex FMHA forward	0.10	0.29	1.14
FLASHATTENTION forward	0.08	0.22	0.81
Apex FMHA backward	0.17	0.52	1.81
FLASHATTENTION backward	0.20	0.53	2.00
Apex FMHA forward + backward	0.27	0.81	2.95
FLASHATTENTION forward + backward	0.28	0.75	2.81

FMHA 面向 BERT 模型，仅支持头维度 64，且只能在 A100 GPU 上运行。FMHA 将注意力计算 $\text{dropout}(\text{softmax}(\text{mask}(\mathbf{QK}^T)))\mathbf{V}$ 融合进一个 CUDA 内核中。在前向传播过程中，它将注意力矩阵 $\text{softmax}(\text{mask}(\mathbf{QK}^T))$ 存储到 HBM 中，供梯度计算使用。因此，它并不能大幅节省内存（尽管对于较短序列而言，内存占用通常不是首要问题）。

我们以 FMHA 代码为起点，应用两种成熟技术（分块和重计算）来处理长序列并节省内存，如第 3 节所述。因此，我们能够支持更长的序列（例如，最长可达 64K）。我们还支持更多的头维度（16、32、64、128）和更广泛的 GPU 类型（撰写本文时所有 Turing 和 Ampere GPU）。

在表7中，我们比较了FlashAttention与Apex FMHA在短序列上的性能（因为FMHA最多支持512的序列长度）。总体而言，FlashAttention在前向传播中比FMHA稍快，而在反向传播中稍慢。这是因为我们在前向传播中不存储注意力矩阵，并在反向传播中重新计算它。与FMHA相比，FlashAttention的整体运行时间在序列长度为128时约慢4%，在序列长度为256时约快8%，在序列长度为512时约快5%。

E.5 在不同硬件和配置上的加速

加速效果因GPU类型和代际不同而异，具体取决于HBM带宽和SRAM大小。本节将分析FlashAttention在不同GPU及配置下的加速情况。

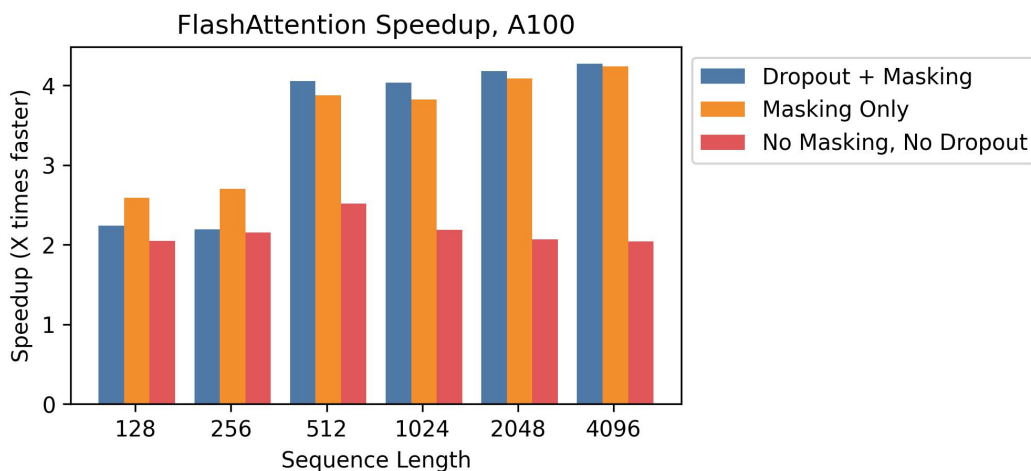


图5：在A100上，不同序列长度下相对于标准PyTorch注意力的加速比。

A100 图5显示了在A100 GPU上的加速情况，批量大小为8，头维度为64，12个注意力头，跨不同序列长度。我们通常观察到2-4×的加速，并且在使用dropout和掩码时由于内核融合而获得更大的加速。

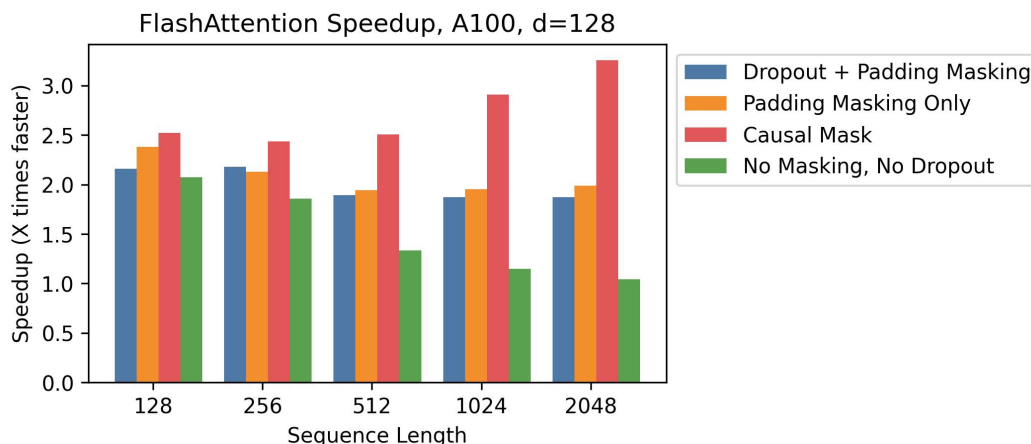


图6：在A100上，不同序列长度下相对于标准PyTorch注意力的加速比（头维度128）。

A100，头部维度128加速比也会随着头部维度的增加而变化。每个块需要更多内存，因此我们需要使用更小的块大小以适应SRAM。图6展示了在A100上头部维度为128时的加速比（批次大小16，12个头部）。整体加速比有所降低——但使用因果掩码时仍能看到显著加速（最高可达3×），其中一半的块被掩码处理。

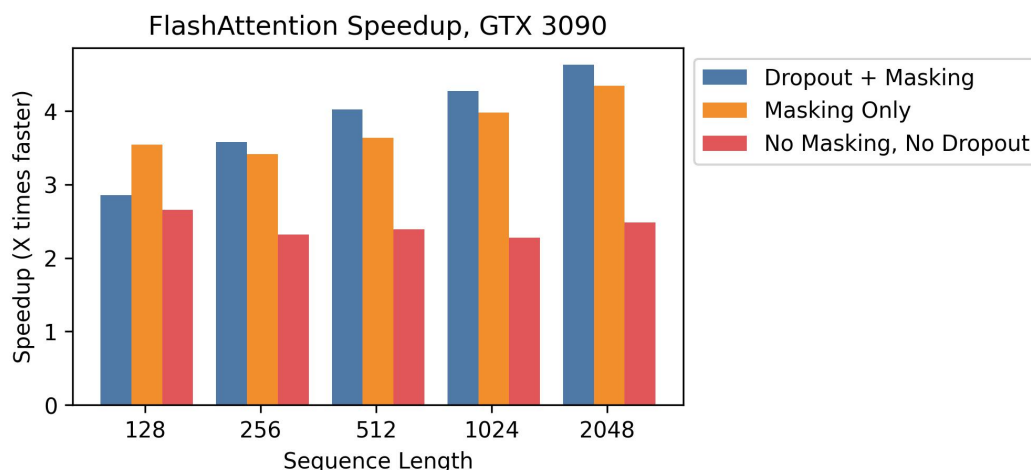


图7：在不同序列长度下，相比标准PyTorch attention的加速比，在RTX 3090上。

RTX 3090 图7展示了在RTX 3090 GPU上的加速比。此处，我们使用批量大小为12以及12个注意力头。我们观察到在RTX 3090上的加速比稍高（介于2.5-4.5×之间），这是因为RTX 3090的内存带宽低于A100（约900 GB/s对比1.5 TB/s）。

T4 图8展示了在T4 GPU上的加速效果。T4的SRAM比A100小，因此我们需要在FlashAttention中使用更小的块大小。因此，我们在T4上观察到的加速效果较小，这与第3.2节中的IO复杂度分析一致。T4 GPU通常用于推理，因此我们也仅报告前向传播的加速效果。

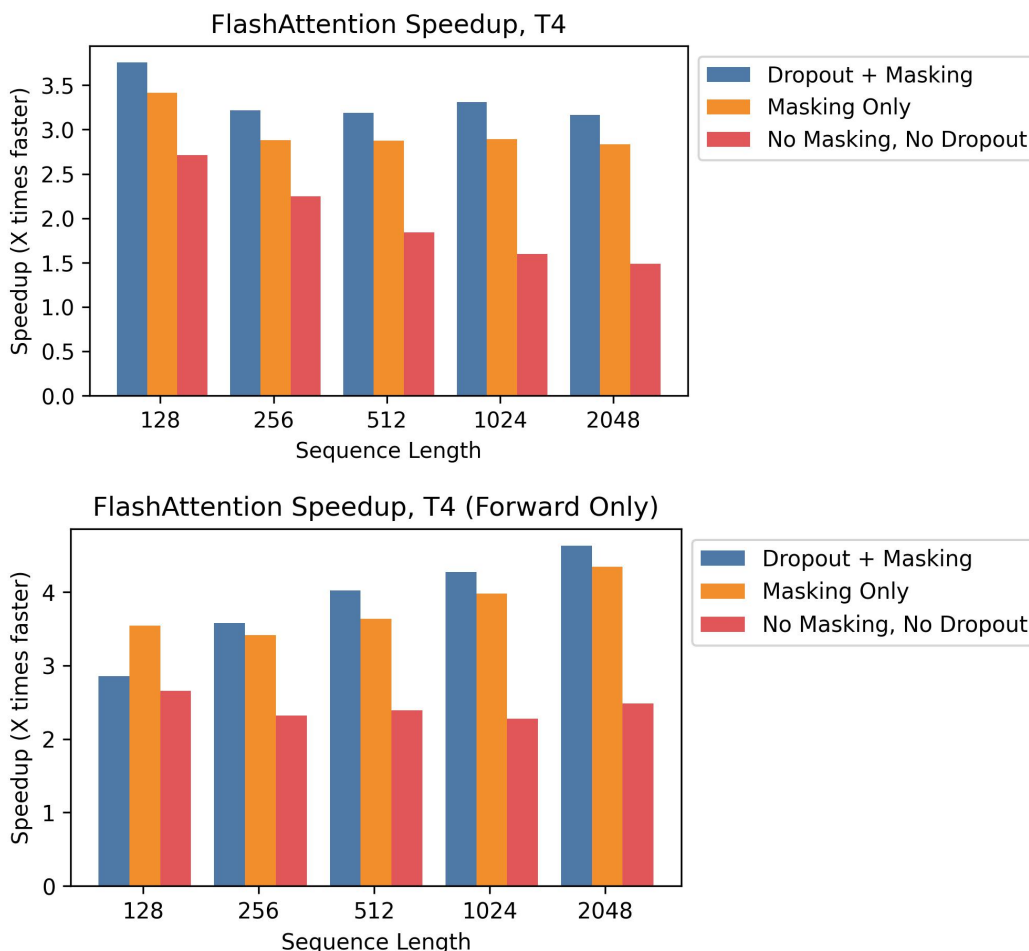


图 8：在 T4 上不同序列长度下相对于标准 PyTorch 注意力的加速比。上方：前向传播 + 反向传播合计。下方：仅前向传播。

E.6 全面基准测试结果

我们报告了在 A100 上的全面基准测试结果和实验细节。

基准方法 我们对比了 PyTorch/HuggingFace 和 Megatron 中的精确注意力、近似注意力以及稀疏注意力的参考实现。对于近似注意力，我们比较了 Reformer [51]、局部注意力 [68]、Linformer 注意力 [84]、Smyrf [19] 和 LongShortFormer (LSFormer) [94] 的参考实现。对于稀疏注意力，我们比较了 OpenAI 的块稀疏注意力 [11]、Longformer [3] 和 BigBird 注意力 [92] 的参考实现。在近似注意力和稀疏注意力中，我们采用 1/8 的压缩比或 256 的压缩序列长度，取两者中较小者。

实验设置 我们在一台配备 40 GB GPU HBM 的 A100 GPU 上，测量了 8 个维度为 64 的注意力头、批量大小为 16 时的注意力计算运行时间与内存占用。实验中我们改变序列长度。我们对随机向量计算 Q 、 K 和 V (的注意力，不测量从隐藏层) 的投影。dropout 率设为 0.1；掩码使用填充掩码，其掩码长度在总序列长度到总序列长度减 20 之间均匀随机选取。为测量运行时间，我们对注意力调用进行了 100 次测量取平均值。内存占用仅测量一次，因为它在不同运行间保持不变。

表8：指向结果表格的指针。

Dropout	Masking	Pass	Table
Yes	Yes	Forward	Table 9
Yes	Yes	Backward	Table 10
Yes	Yes	Combined	Table 11
No	Yes	Forward	Table 12
No	Yes	Backward	Table 13
No	Yes	Combined	Table 14
Yes	No	Forward	Table 15
Yes	No	Backward	Table 16
Yes	No	Combined	Table 17
No	No	Forward	Table 18
No	No	Backward	Table 19
No	No	Combined	Table 20
No	No	Memory Usage (Combined)	Table 21

表9：不同序列长度下各类精确/近似/稀疏注意力机制的前向传播耗时（毫秒），包含dropout与掩码处理。最优结果加粗，次优加下划线。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
Local Attention	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer	0.67	0.52	0.69	<u>0.71</u>	<u>1.65</u>	<u>3.18</u>	<u>6.15</u>	<u>12.16</u>	<u>24.17</u>	<u>52.39</u>
Smyrf	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
Block Sparse	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FLASHATTENTION	0.04	<u>0.06</u>	<u>0.21</u>	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
Block-Sparse FLASHATTENTION	<u>0.06</u>	0.06	0.06	0.12	0.44	0.86	1.70	3.29	6.55	13.34

我们报告了前向传播、反向传播以及前向+反向传播组合的计时结果。我们对每种方法在有 dropout、masking 或两者兼有的情况下进行测量——但 Block Sparse、Longformer 和 BigBird 除外。这些方法因外部库中的错误未能成功运行带 masking 的反向传播，因此为公平起见，我们在不带 masking 的情况下对它们进行了测量。我们所有测量均使用 FP16，但 Local Attention 除外，因其实现仅支持 FP32。

对于每种基线，我们不断增加序列长度，直到其因GPU内存不足而无法运行为止，但有以下例外：Megatron实现不支持超过2048的序列长度。Block-Sparse（OpenAI）不支持超过4096的序列长度。Longformer和BigBird不支持超过8092的序列长度。

我们测量了组合前向+后向传递中的内存使用情况，未使用 dropout 或掩码。

结果表8汇总了所有实验配置，并包含指向各结果表的索引。

表10：各种精确/近似/稀疏注意力机制在不同序列长度下的反向传播运行时间（毫秒），包含dropout和掩码。最优值以粗体表示，次优值以下划线表示。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
Local Attention	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	<u>36.53</u>	-
Smyrf	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
Block Sparse	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FLASHATTENTION	0.30	0.26	<u>0.68</u>	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
Block-Sparse FLASHATTENTION	0.30	<u>0.27</u>	0.29	0.59	1.50	2.94	5.82	11.85	23.98	47.61

表11：各种精确/近似/稀疏注意力机制在不同序列长度下的前向传递+反向传递运行时间（毫秒），含dropout和掩码。最佳结果以粗体标出，次佳结果以下划线标出。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
Local Attention	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	<u>61.47</u>	-
Smyrf	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FLASHATTENTION	0.43	0.41	<u>0.95</u>	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
Block-Sparse FLASHATTENTION	<u>0.44</u>	<u>0.44</u>	0.45	0.89	1.95	4.12	7.64	16.60	32.73	64.11

表12：不同精确/近似/稀疏注意力机制在按序列长度划分的前向传播运行时间（毫秒），含掩码。最优以粗体标出，次优以下划线标出。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
Local Attention	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer	0.63	0.50	0.67	<u>0.65</u>	<u>1.36</u>	<u>2.60</u>	<u>5.04</u>	<u>9.92</u>	<u>19.69</u>	<u>43.47</u>
Smyrf	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FLASHATTENTION	0.03	0.04	<u>0.17</u>	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
Block-Sparse FLASHATTENTION	<u>0.05</u>	0.04	0.05	0.11	0.35	0.68	1.33	2.54	5.34	10.73

表13：各种精确/近似/稀疏注意力机制按序列长度划分的前向传播运行时间（毫秒），带掩码。最佳者以粗体显示，次佳者以下划线显示。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
Local Attention	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	<u>32.67</u>	<u>72.11</u>
Smyrf	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FLASHATTENTION	0.21	0.22	<u>0.62</u>	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
Block-Sparse FLASHATTENTION	<u>0.22</u>	<u>0.22</u>	0.26	0.57	1.55	3.13	5.98	12.21	23.49	47.85

表14: 不同序列长度下, 各种精确/近似/稀疏注意力机制在采用掩码时的前向传播 + 后向传播耗时 (毫秒)。最优结果加粗, 次优结果加下划线。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
Local Attention	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	<u>53.36</u>	<u>117.56</u>
Smyrf	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FLASHATTENTION	0.32	0.30	<u>0.83</u>	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
Block-Sparse FLASHATTENTION	<u>0.34</u>	<u>0.34</u>	0.36	0.69	1.85	3.89	7.16	14.85	30.46	60.03

表15: 各种精确/近似/稀疏注意力机制在不同序列长度下的前向传播运行时间 (毫秒), 带有dropout。最优用粗体表示, 次优用下划线表示。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
Local Attention	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer	0.47	0.37	0.49	0.52	1.37	2.65	5.12	10.13	<u>20.25</u>	<u>44.16</u>
Smyrf	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
Block Sparse	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FLASHATTENTION	0.10	0.10	0.22	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
Block-Sparse FLASHATTENTION	0.54	0.51	0.68	<u>0.61</u>	0.67	1.10	1.89	3.71	7.18	14.41

表 16: 按序列长度划分的各种精确/近似/稀疏注意力机制在包含 dropout 时的反向传播运行时间 (毫秒)。最佳结果加粗, 次佳结果加下划线。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
Local Attention	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer	0.84	0.74	0.79	<u>0.85</u>	<u>2.28</u>	<u>4.37</u>	<u>8.66</u>	<u>17.02</u>	<u>33.78</u>	-
Smyrf	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
Block Sparse	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FLASHATTENTION	0.15	<u>0.18</u>	<u>0.58</u>	1.86	6.50	26.21	104.27	416.10	1661.92	<u>6643.01</u>
Block-Sparse FLASHATTENTION	<u>0.17</u>	0.17	0.17	0.40	1.10	2.04	4.43	9.33	18.28	37.31

表17: 不同序列长度下含dropout的各种精确/近似/稀疏注意力机制的前向传播+反向传播运行时间 (毫秒)。最优结果以粗体标示, 次优结果以下划线标示。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.66	<u>0.67</u>	1.43	4.82	17.47	67.29	-	-	-	-
Megatron	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
Local Attention	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	<u>55.00</u>	-
Smyrf	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FLASHATTENTION	0.35	0.36	0.80	2.52	9.16	36.70	146.13	583.45	2332.01	<u>9323.63</u>
Block-Sparse FLASHATTENTION	0.91	0.83	<u>0.94</u>	0.92	1.83	3.50	7.02	13.56	26.71	53.92

表18：不同序列长度下各种精确/近似/稀疏注意力机制的前向传播运行时间（毫秒）。最佳结果加粗，次优结果加下划线。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	<u>0.21</u>	<u>0.22</u>	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron	0.24	0.26	<u>0.42</u>	1.33	4.28	-	-	-	-	-
Reformer	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
Local Attention	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer	0.46	0.36	0.45	0.50	<u>1.09</u>	<u>2.09</u>	<u>4.01</u>	<u>7.90</u>	<u>15.70</u>	<u>35.40</u>
Smyrf	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FLASHATTENTION	0.08	0.09	0.18	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
Block-Sparse FLASHATTENTION	0.56	0.52	0.63	<u>0.65</u>	0.61	0.96	1.69	3.02	5.69	11.77

表19：不同精确/近似/稀疏注意力机制按序列长度的反向传播运行时间（毫秒）。最佳加粗，次佳加下划线。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
Local Attention	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer	0.69	0.76	0.69	0.80	2.04	3.88	<u>7.67</u>	<u>15.04</u>	<u>30.11</u>	<u>63.15</u>
Smyrf	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FLASHATTENTION	0.11	<u>0.16</u>	<u>0.52</u>	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
Block-Sparse FLASHATTENTION	<u>0.11</u>	0.12	0.16	0.38	1.20	2.34	4.69	9.10	18.74	37.04

表20：各种精确/近似/稀疏注意力机制在不同序列长度下的前向+后向传递运行时间（毫秒）。最佳值以粗体显示，次佳值以下划线显示。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron	0.74	<u>0.65</u>	1.23	3.80	13.21	-	-	-	-	-
Reformer	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
Local Attention	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer	1.31	1.25	1.30	<u>1.29</u>	<u>3.20</u>	<u>6.10</u>	<u>11.93</u>	<u>23.39</u>	<u>46.72</u>	<u>100.52</u>
Smyrf	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FLASHATTENTION	0.31	0.31	0.73	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
Block-Sparse FLASHATTENTION	0.74	0.77	<u>0.82</u>	0.88	1.71	3.21	6.56	12.60	24.93	50.39

表21：不同精确/近似/稀疏注意力机制在不同序列长度下的内存使用量（MB）。最佳值以粗体标出，次佳值以下划线标出。

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	36	104	336	1184	4416	17024	-	-	-	-
Megatron	36	104	336	1184	4416	-	-	-	-	-
Reformer	377	754	1508	3016	6033	12067	24134	-	-	-
Local Attention	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer	25	52	114	287	832	1652	3292	6572	13132	26252
Smyrf	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer	72	152	333	796	2540	5068	10125	20240	-	-
Block Sparse	33	82	228	408	910	2401	-	-	-	-
Longformer	30	61	124	277	681	1370	2748	-	-	-
BigBird	33	66	131	294	708	1431	2872	-	-	-
FLASHATTENTION	22	44	104	209	418	836	1672	3344	6688	13376
Block-Sparse FLASHATTENTION	<u>22</u>	<u>44</u>	<u>104</u>	<u>209</u>	<u>418</u>	<u>836</u>	<u>1672</u>	<u>3344</u>	<u>6690</u>	<u>13384</u>